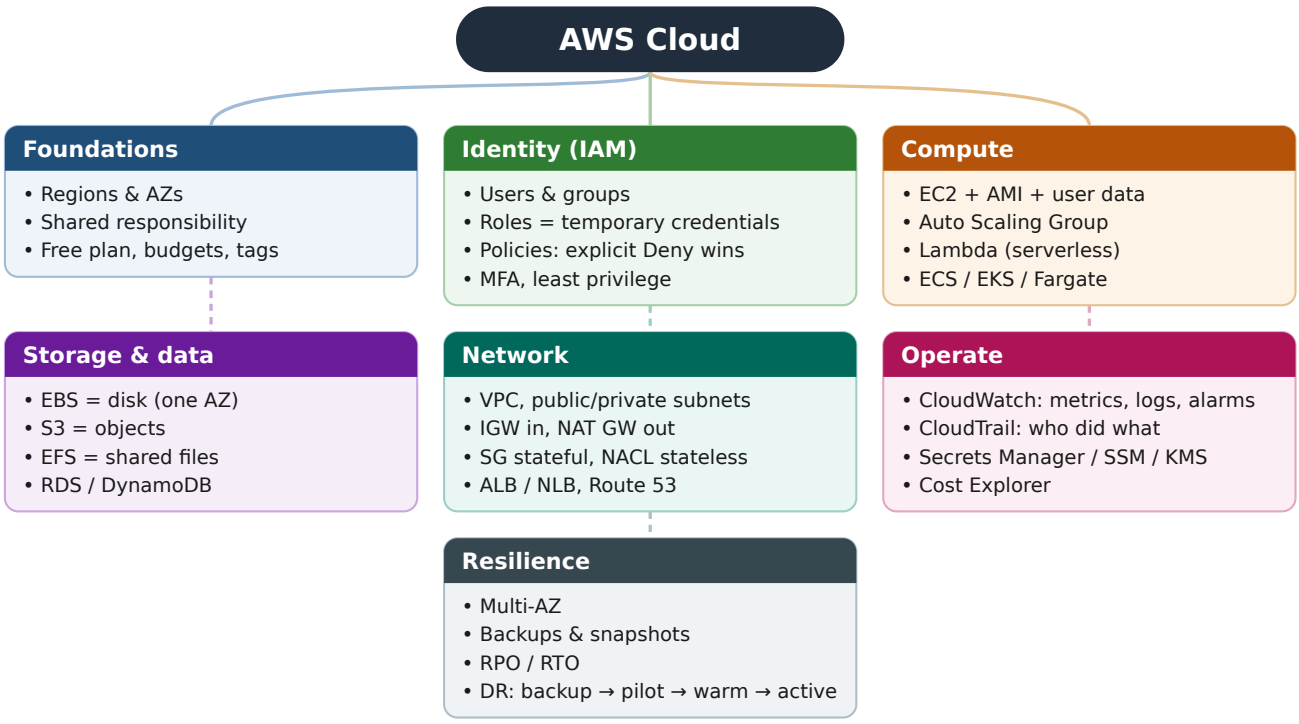
# 7. AWS Cloud

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 7 of 18 · Thirupathi Keloth · v2.0

## Contents


7.0 How to use these notes
7.1 Cloud computing fundamentals
7.2 AWS global infrastructure
7.3 Shared responsibility model
7.4 How AWS pricing works
7.5 Ways to use AWS
7.6 Account setup and security baseline
7.7 IAM: Identity and Access Management
7.8 AWS CLI in depth
7.9 EC2: Elastic Compute Cloud
7.10 Storage for EC2: EBS, instance store, EFS
7.11 S3: Simple Storage Service
7.12 VPC: Virtual Private Cloud
7.13 Elastic Load Balancing (ELB)
7.14 EC2 Auto Scaling
7.15 Route 53 (DNS)
7.16 CloudFront and Global Accelerator
7.17 Databases on AWS
7.18 Application integration (decoupling services)
7.19 Analytics services (awareness level)
7.20 Serverless: Lambda and API Gateway
7.21 Containers on AWS
7.22 Other compute options
7.23 Management, operations and Infrastructure as Code
7.24 Monitoring and auditing
7.25 Security services and data protection
7.26 Multi-account governance
7.27 Cost management (FinOps basics)
7.28 High availability, backup and disaster recovery
7.29 Migration to AWS (awareness)
7.30 Well-Architected Framework
7.31 Reference architectures
7.32 AWS troubleshooting playbook
7.33 Interview questions and answers
7.34 Self-check: I can…

## 7.0 How to use these notes

- **Prerequisites:** Networking (IP, subnets, DNS, ports, TCP/HTTP) and Linux basics. AWS builds on both.
- **Order:** read the sections in order the first time. Later, use them as reference.
- **Each service is explained the same way:** what it is → why we use it → how it works → key settings → commands → best practices → common mistakes.
- **Labs** are marked ⚠️ when they can use credits. Every lab ends with **cleanup**. Always check **Billing → Bills** after a lab.
- **Region used in examples:** `ap-south-1` (Mumbai). Replace it with your own region.
- Placeholders look like `<your-bucket-name>` or `i-0123456789abcdef0`: replace them with your real values.

## 7.1 Cloud computing fundamentals

### What cloud computing is

**What:** renting IT resources (servers, storage, databases, networks, software) over the internet, **on demand**, and **paying only for what you use**.

**Why companies use it:**

| Traditional data center | Cloud |
|---|---|
| Buy servers months in advance (big upfront cost, CapEx) | Pay monthly for what you use (operating cost, OpEx) |
| Guess capacity: too much = waste, too little = outage | Scale up and down in minutes |
| Weeks to get a new server | Minutes to get a new server |
| You run the building, power, cooling, hardware | The provider does |
| Expanding to another country = building a data center | Deploy to another region in minutes |

AWS lists six benefits: trade upfront expense for variable expense · benefit from massive economies of scale · stop guessing capacity · increase speed and agility · stop spending money running data centers · go global in minutes.

**Real-world example:** an online shop needs 10 servers normally and 100 during a sale. In the cloud it runs 10, automatically grows to 100 for the sale, and goes back to 10 afterwards, paying for the extra servers only for those hours.

### Service models

```
                Applications   Data   Runtime   OS   Virtualization   Servers   Storage   Network
On-premises     YOU            YOU    YOU       YOU  YOU              YOU       YOU       YOU
IaaS  (EC2)     YOU            YOU    YOU       YOU  AWS              AWS       AWS       AWS
PaaS  (RDS)     YOU            YOU    AWS       AWS  AWS              AWS       AWS       AWS
SaaS  (Gmail)   provider       YOU*   provider  ...  provider         provider  provider  provider
                               * you still own your data and user access
```

| Model | You get | You manage | AWS examples |
|---|---|---|---|
| IaaS (Infrastructure as a Service) | Virtual servers, networks, disks | OS patches, software, data, security config | EC2, EBS, VPC |

| Model | You get | You manage | AWS examples |
|---|---|---|---|
| PaaS (Platform as a Service) | A platform to run code or databases | Your code and data | Elastic Beanstalk, RDS, Lambda (serverless) |
| SaaS (Software as a Service) | Finished software | Users and data | Amazon WorkMail, Chime (and non-AWS: Gmail) |

### Deployment models

| Model | Meaning | Example |
|---|---|---|
| Public cloud | Resources run on a provider's shared infrastructure | A startup fully on AWS |
| Private cloud | Cloud-style infrastructure dedicated to one organization (often on-premises) | A bank running OpenStack/VMware in its own data center |
| Hybrid cloud | On-premises connected to public cloud | Core banking on-premises, website and analytics on AWS, connected by VPN/Direct Connect |
| Multi-cloud | More than one public cloud provider | AWS for apps, Google Cloud for analytics |

## 7.2 AWS global infrastructure

```
AWS GLOBAL NETWORK
│
├── Region  ap-south-1 (Mumbai)              ← a geographic area, fully independent
│     ├── Availability Zone ap-south-1a       ← one or more data centers
│     ├── Availability Zone ap-south-1b       ← separate power, cooling, network
│     └── Availability Zone ap-south-1c       ← connected by fast, low-latency private links
│
├── Region  us-east-1 (N. Virginia) ...
│
├── Edge locations (hundreds of cities)       ← CloudFront CDN, Route 53 DNS
├── Local Zones                               ← compute close to big cities (very low latency)
├── Wavelength Zones                          ← compute inside 5G telecom networks
└── Outposts                                  ← AWS hardware installed in YOUR data center
```

| Component | What it is | Why it matters |
|---|---|---|
| **Region** | A geographic area with multiple AZs (e.g. `ap-south-1`) | Data stays in the region you choose (laws, latency). Most services are **regional** |
| **Availability Zone (AZ)** | One or more physically separate data centers in a region (e.g. `ap-south-1a`) | Spread your app across **2+ AZs** so one data center failure doesn't take you down |
| **Edge location** | Small sites near users | CloudFront caches content and Route 53 answers DNS close to users |
| **Local Zone / Wavelength / Outposts** | Extensions of a region | Very low latency or on-premises needs |

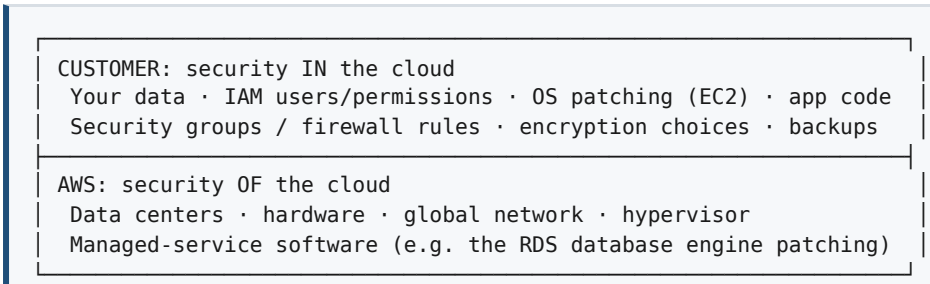
**How to choose a region:** 1. **Compliance / data laws:** e.g. data must stay in India → `ap-south-1` or `ap-south-2`. 2. **Latency:** close to your users. 3. **Service availability:** new services don't launch in every region at once. 4. **Price:** prices differ between regions (`us-east-1` is often cheapest).

**Global vs regional vs zonal services:**

| Scope | Examples |
|---|---|
| Global | IAM, Route 53, CloudFront, Organizations |
| Regional | S3 buckets, VPC, Lambda, DynamoDB, most services |
| Zonal (AZ-specific) | EC2 instances, EBS volumes, subnets |

**Common mistake:** "My EC2 instances disappeared!" → you are looking at a different region in the console (top-right region selector).

## 7.3 Shared responsibility model




The line moves depending on the service:

| Task | EC2 (IaaS) | RDS (managed DB) | Lambda (serverless) | S3 |
|---|---|---|---|---|
| Physical security, hardware | AWS | AWS | AWS | AWS |
| Operating system patches | **You** | AWS | AWS | AWS |
| Database engine patches | **You** (if installed) | AWS (you choose the window) | — | — |
| Network firewall rules (SGs) | **You** | **You** | **You** (if in a VPC) | — |
| Access permissions (IAM, policies) | **You** | **You** | **You** | **You** |
| Data encryption & backups config | **You** | **You** | **You** | **You** |

**Interview line:** "AWS is responsible for security *of* the cloud; the customer is responsible for security *in* the cloud. The more managed the service, the more AWS takes on, but data and access control are always the customer's."

## 7.4 How AWS pricing works

**Three main cost drivers: compute** (per second/hour), **storage** (per GB-month), **data transfer** (per GB).

| Data transfer | Typical cost |
|---|---|
| Into AWS from the internet | Free |
| Out of AWS to the internet | Charged per GB (the first 100 GB/month is free across all services) |
| Between AZs in the same region | Charged (small, per GB, each direction) |

| Data transfer | Typical cost |
|---|---|
| Between regions | Charged per GB |
| Within the same AZ using private IPs | Free |

**Purchasing options (compute):**

| Option | How it works | Saving vs On-Demand | Use for |
|---|---|---|---|
| On-Demand | Pay per second, no commitment | — | Dev/test, unpredictable workloads, short labs |
| Savings Plans | Commit to $X/hour for 1 or 3 years | Up to ~72% | Steady production usage |
| Reserved Instances | Commit to a specific instance type/region | Up to ~72% | Older model, still common |
| Spot Instances | Use spare capacity; AWS can reclaim with a **2-minute warning** | Up to ~90% | Batch jobs, CI runners, stateless workers |
| Dedicated Hosts/Instances | Hardware not shared with other customers | More expensive | Licensing or compliance |

**AWS Free plan (accounts created since July 2025):** $100 credits at sign-up + up to $100 more for completing onboarding activities; the plan lasts **6 months or until credits run out**. Some services always have a free amount (e.g. Lambda requests, DynamoDB storage). At the end you choose to upgrade to a paid plan; otherwise the account is closed. Always confirm the current rules on the AWS Free Tier page.

**Always-on cost traps:** NAT Gateways, load balancers, public IPv4 addresses (billed per hour), EKS control planes, RDS instances, unattached EBS volumes, old snapshots, interface VPC endpoints, resources left in another region.

## 7.5 Ways to use AWS

| Method | What it is | Used for |
|---|---|---|
| Management Console | Web interface | Learning, exploring, one-off checks |
| AWS CLI | Command-line tool ( `aws ...` ) | Scripts, automation, fast admin work |
| SDKs | Libraries for Python (boto3), JavaScript, Go, Java... | Applications and tools that call AWS |
| Infrastructure as Code | Terraform, CloudFormation, CDK | **How companies build production infrastructure** |
| CloudShell | Browser terminal with the CLI pre-installed and already logged in | Quick CLI work without local setup |

### ARN (Amazon Resource Name)

Every AWS resource has a unique ID called an **ARN**:

```
arn:partition:service:region:account-id:resource
arn:aws:s3:::my-bucket                                  ← S3 (global name: no region/account)
arn:aws:ec2:ap-south-1:123456789012:instance/i-0abc123  ← EC2 instance
arn:aws:iam::123456789012:role/web-server-role          ← IAM (global: no region)
```

ARNs are used in IAM policies, CLI commands and Terraform outputs.

### Tags

**Tags** are key-value labels on resources: `Project=shop`, `Env=prod`, `Owner=thirupathi`, `CostCenter=1234`. **Why:** find resources, split costs per team/project (cost allocation tags), automate (e.g. "stop all instances with `AutoStop=true` at night"), and control access with tag-based IAM conditions. **Best practice:** agree on a tagging standard and enforce it (Terraform `default_tags`, AWS Organizations tag policies).

## 7.6 Account setup and security baseline

Do this **once, on day one** of any AWS account.

### Step-by-step

1. **Create the account** at aws.amazon.com (email, account name, card for verification, phone check, choose the **Free plan**).
2. **Protect the root user** (the email you signed up with = full, unlimited access): - Enable **MFA** on root (IAM → Security credentials → Assign MFA; use an authenticator app or hardware key). - Never create access keys for root. - Store the root password safely and **stop using root** for daily work.
3. **Create your daily admin identity:** - Recommended: **IAM Identity Center** (single sign-on) user with the `AdministratorAccess` permission set, then log in through the access portal URL. - Simpler for a personal lab: an **IAM user** in an `Admins` group with `AdministratorAccess`, console password and **MFA**.
4. **Billing safety:** - Billing → **Budgets** → create a monthly cost budget (e.g. $10) with email alerts at 50%, 80%, 100%. - Turn on **Free plan usage alerts** and check the credit balance page. - Enable **IAM user access to billing** (Account settings) if you use an IAM admin user.
5. **Visibility:** CloudTrail **Event history** is on by default (90 days of management events). For long-term audit, create a **trail** that writes to S3.
6. **Pick one region** and use it consistently.
7. **Set the account alternate contacts** (billing, operations, security).

### Tasks only the root user can do

Change account settings (name, email, root password), close the account, change the support plan, restore IAM admin permissions if they were removed, some billing tasks. Everything else: use your admin identity.

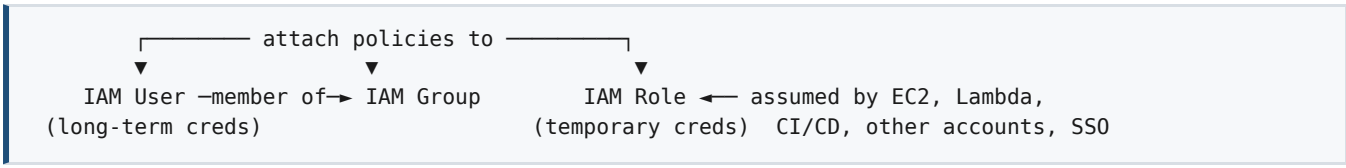
## 7.7 IAM: Identity and Access Management

**What:** the AWS service that controls **who** can do **what** on **which** resources. **Why:** every API call to AWS is checked by IAM. Most cloud breaches come from leaked credentials or permissions that are too broad, so IAM is the most important security service. **Scope:** global (not per region), free.

### Core concepts

| Concept | What it is | Example |
|---|---|---|
| **Principal** | Who is making the request | A user, a role session, an AWS service |
| **Authentication** | Proving who you are | Password + MFA, access keys, temporary credentials |
| **Authorization** | Checking what you are allowed to do | IAM evaluates policies |

| Concept | What it is | Example |
|---|---|---|
| **IAM user** | A long-term identity for a person or application, with a password and/or access keys | `thirupathi-admin` |
| **IAM group** | A collection of users that share permissions (groups can't contain groups) | `Developers`, `Admins` |
| **IAM role** | An identity with permissions but **no long-term credentials**. It is **assumed** and gives **temporary credentials** | `web-server-role` used by EC2, `github-deploy-role` used by CI |
| **Policy** | A JSON document that allows or denies actions | `AmazonS3ReadOnlyAccess` |




## Policy anatomy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadAppBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::shop-assets",
        "arn:aws:s3:::shop-assets/*"
      ],
      "Condition": {
        "IpAddress": { "aws:SourceIp": "203.0.113.0/24" }
      }
    }
  ]
}
```

| Element | Meaning |
|---|---|
| `Version` | Policy language version. Always `"2012-10-17"` |
| `Statement` | List of permission rules |
| `Sid` | Optional label for the statement |
| `Effect` | `Allow` or `Deny` |
| `Action` | API actions, format `service:Action`. Wildcards allowed: `s3:Get*`, `ec2:*` |
| `Resource` | ARNs the actions apply to. `*` = all |
| `Condition` | Optional extra checks: source IP, MFA, tags, time, region… |
| `Principal` | Only in **resource-based** policies (who is allowed), e.g. in bucket policies and role trust policies |

**Note:** `s3:ListBucket` applies to the bucket ARN; `s3:GetObject` applies to objects (`bucket/*`). Forgetting one of them is a classic mistake.

## Policy types

| Type | Attached to | Purpose |
|---|---|---|
| AWS managed policy | Users/groups/roles | Ready-made by AWS (e.g. `ReadOnlyAccess`). Broad; good for starting |
| Customer managed policy | Users/groups/roles | Your own reusable policy. **Preferred for least privilege** |
| Inline policy | One user/group/role only | Strict one-to-one; harder to manage |
| Resource-based policy | A resource (S3 bucket, SQS queue, KMS key, role trust policy) | Says **who** may access this resource, including other accounts |
| Permissions boundary | A user or role | The **maximum** permissions it can ever have |
| Service Control Policy (SCP) | AWS Organizations account/OU | Guardrail for entire accounts (never grants, only limits) |
| Session policy | A temporary session | Further limits a role session |

## How AWS decides: policy evaluation

```
Request arrives
   │
   ├─ Is there an explicit DENY in any applicable policy? ── yes ──► DENIED
   │
   ├─ Do SCPs (if any) allow it? ──────────────────────────── no ──► DENIED
   │
   ├─ Is there an ALLOW in identity or resource policy? ──── no ──► DENIED (default deny)
   │
   ├─ Do permission boundary / session policy (if any) allow it? ── no ──► DENIED
   │
   └─ ALLOWED
```

**Remember:** everything is **denied by default** → an **Allow** is needed → an explicit **Deny always wins**.

## IAM roles and STS in depth

A role has **two** policies:

| Policy | Question it answers | Example |
|---|---|---|
| **Trust policy** | **Who** may assume this role? | "The EC2 service may assume me" |
| **Permissions policy** | **What** can the role do once assumed? | "Read objects from bucket `shop-assets`" |

**Trust policy for EC2:**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

**Trust policy for GitHub Actions (OIDC, no stored keys):**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:<owner>/<repo>:ref:refs/heads/main"
      }
    }
  }]
}
```

**How a role is used (STS = Security Token Service):**

```
EC2 instance / Lambda / CI job / user
        │ 1. "I want to assume role X"  (sts:AssumeRole)
        ▼
       STS ── 2. checks the role's TRUST policy
        │ 3. returns temporary credentials (key + secret + session token; 1 hour by default)
        ▼
  App calls AWS APIs with those credentials → checked against the role's PERMISSIONS policy
```

| Role use case | How |
|---|---|
| App on EC2 needs S3 | Role attached through an **instance profile**; SDK/CLI on the instance gets credentials automatically from instance metadata |
| Lambda function | **Execution role** |
| ECS task | **Task role** (app permissions) + **task execution role** (pull image, write logs) |
| EKS pod | IRSA or EKS Pod Identity |
| CI/CD pipeline | OIDC federation → `AssumeRoleWithWebIdentity` |
| Another AWS account | Cross-account role trusting that account |
| People (SSO) | IAM Identity Center permission sets (roles behind the scenes) |
| AWS services themselves | **Service-linked roles** (created automatically, e.g. for Auto Scaling) |

**Why roles beat access keys:** temporary credentials expire automatically, nothing is stored in code or on disk, rotation is automatic, every session is logged in CloudTrail.

## Credentials and authentication hygiene

| Item | Practice |
|---|---|
| MFA | Required for root and all human users |
| Access keys | Avoid. If required: one per user, rotate, never in code/Git, delete unused ones |
| Password policy | Length, complexity, rotation as company policy requires |

| Item | Practice |
|---|---|
| Credential report | IAM → Credential report: lists all users, key ages, MFA status |
| Last accessed info | Shows which services a role/user actually used → remove the rest |
| IAM Access Analyzer | Finds resources shared publicly or with other accounts; generates least-privilege policies from CloudTrail activity |

## IAM Identity Center (SSO)

Central login for people across one or many AWS accounts. Users sign in to an **access portal**, pick an account and a **permission set**, and get temporary credentials for the console and CLI ( `aws configure sso` ). Can connect to an external identity provider (Microsoft Entra ID, Okta, Google). **This is how most companies give engineers AWS access today.**

## Best practices

1. Lock down root (MFA, no keys, don't use it).
2. Humans use **SSO/federation**; workloads use **roles**. Avoid long-term access keys.
3. **Least privilege:** start small, add permissions when needed; use Access Analyzer to trim.
4. Use **groups** (or permission sets), not per-user policies.
5. Use **conditions** (MFA required, source IP, tags, region).
6. Separate environments with **separate accounts** (dev/prod) under Organizations.
7. Monitor with CloudTrail; alert on root logins and IAM changes.

## Hands-on: least-privilege access (free)

**Goal:** a `Developers` group that can only read one S3 bucket, and an EC2 role that can read the same bucket.

1. **Policy:** IAM → Policies → Create policy → JSON → paste the "Policy anatomy" example above (without the `Condition` ) using your bucket name → name it `ShopAssetsRead` .
2. **Group:** IAM → User groups → Create `Developers` → attach `ShopAssetsRead` .
3. **User:** IAM → Users → Create `dev-test` → console access → add to `Developers` .
4. **Test:** log in as `dev-test` (private browser window) → S3 → you can open `shop-assets` but get **Access Denied** on other buckets and on EC2. This is least privilege working.
5. **Role:** IAM → Roles → Create role → Trusted entity **AWS service → EC2** → attach `ShopAssetsRead` → name `web-server-role` . (You'll attach it to an instance in the EC2 lab.)

**Same with the CLI:**

```
aws iam create-policy --policy-name ShopAssetsRead --policy-document file://shop-read.json
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::<account-id>:policy/ShopAssetsRead
aws iam create-role --role-name web-server-role --assume-role-policy-document file://ec2-trust.json
aws iam attach-role-policy --role-name web-server-role \
  --policy-arn arn:aws:iam::<account-id>:policy/ShopAssetsRead
aws iam create-instance-profile --instance-profile-name web-server-role
aws iam add-role-to-instance-profile --instance-profile-name web-server-role \
  --role-name web-server-role
```

**Cleanup:** delete the user, group, role, instance profile and policy when finished.

## Troubleshooting "AccessDenied"

1. **Who am I?** `aws sts get-caller-identity` (right account? right role?).

2. **Read the error message:** it names the action and resource (e.g. `not authorized to perform: s3:GetObject on resource: arn:aws:s3:::bucket/file`).
3. Is there an **Allow** for that exact action **and** resource ARN (bucket vs `bucket/*`)?
4. Is there an explicit **Deny** anywhere (identity policy, bucket policy, SCP, permission boundary)?
5. **Resource policy:** bucket/KMS key policies can block access even when IAM allows it (e.g. encrypted object without `kms:Decrypt`).
6. **Conditions:** MFA required? Wrong source IP or region? Missing tag?
7. Use the **IAM policy simulator** and **CloudTrail** (the event shows the exact denied call).

## 7.8 AWS CLI in depth

### Install and configure

- Install AWS CLI v2 (Windows MSI / `curl` installer on Linux / `brew` on macOS). Check: `aws --version`.
- **Recommended login (SSO):** `aws configure sso` → enter the SSO start URL and region → pick account and role → profile name. Later: `aws sso login --profile dev`.
- **Access-key login (labs only):** `aws configure` → access key, secret key, default region, output format.

**Where settings live:**

| File | Contains |
|---|---|
| `~/.aws/config` | Profiles, regions, output format, SSO settings |
| `~/.aws/credentials` | Access keys (if used). **Never commit or share this file** |

**Which credentials win (simplified order):** command-line options → environment variables (`AWS_ACCESS_KEY_ID`, `AWS_PROFILE`, `AWS_REGION`) → SSO / `credentials` file → container credentials → EC2 instance role.

### Command structure

```
aws  ec2  describe-instances  --region ap-south-1  --output table
|    |    |                   └──────────── options ────────────┘
|    |    └ operation (verb-noun)
|    └ service
└ the CLI
```

### Output and filtering

| Option | Use |
|---|---|
| `--output json \| table \| text \| yaml` | Output format (`text` is best for scripts) |
| `--query '<JMESPath>'` | Filter the output **on your side** |
| `--filters Name=...,Values=...` | Filter **on the AWS side** (supported by many describe commands) |
| `--profile`, `--region` | Choose account/role and region |
| `--dry-run` | EC2: check permissions without doing anything |
| `--no-cli-pager` | Don't open output in a pager |

**JMESPath examples:**

```
# IDs of all instances
aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId' --output text

# Table of ID, state and Name tag
aws ec2 describe-instances --output table --query \
  'Reservations[].Instances[].{ID:InstanceId,State:State.Name,Name:Tags[?Key==`Name`]|[0].Value}'

# Only running instances (filter on the AWS side)
aws ec2 describe-instances --filters Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[].[InstanceId,PrivateIpAddress]' --output text

# Wait until an instance is running
aws ec2 wait instance-running --instance-ids i-0123456789abcdef0
```

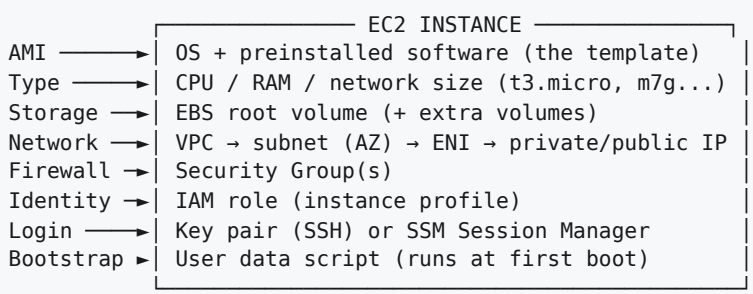
### Commands you'll use constantly

| Command | Purpose |
|---|---|
| `aws sts get-caller-identity` | Which account and identity am I using? |
| `aws configure list` / `aws configure list-profiles` | Current settings / available profiles |
| `aws ec2 describe-regions --output table` | List regions |
| `aws ec2 describe-instances` | List instances |
| `aws s3 ls` | List buckets |
| `aws iam list-roles --query 'Roles[].RoleName'` | List roles |
| `aws logs tail /aws/lambda/my-fn --follow` | Follow CloudWatch logs live |
| `aws ssm start-session --target i-0123...` | Shell on an instance without SSH (needs the Session Manager plugin) |
| `aws <service> help` | Built-in help for any service |

## 7.9 EC2: Elastic Compute Cloud

**What:** virtual servers ("instances") in the cloud. **Why:** full control over the OS and software, any size from tiny to huge, pay per second, start/stop in minutes. **In companies:** web and app servers, CI runners, bastion hosts, legacy apps, Kubernetes worker nodes (EKS nodes are EC2).

### Building blocks of an instance




### Instance types

**Reading a name:** `m7g.xlarge`

| Part | Meaning |
|---|---|
| `m` | **Family** (purpose) |
| `7` | **Generation** (newer = better price/performance) |
| `g` | **Attributes**: `g` = AWS Graviton (ARM), `a` = AMD, `i` = Intel, `d` = local NVMe disk, `n` = more network, `e` = extra memory |
| `xlarge` | **Size**: nano, micro, small, medium, large, xlarge, 2xlarge… (each step roughly doubles CPU/RAM) |

| Family | Optimized for | Examples | Typical use |
|---|---|---|---|
| T (burstable) | Low baseline CPU with bursts (CPU credits) | t3.micro, t4g.small | Dev/test, small websites, labs |
| M (general purpose) | Balanced CPU/RAM | m7i.large, m7g.large | Most app servers |
| C (compute) | High CPU | c7g.xlarge | Batch processing, CI builds, high-traffic web |
| R / X (memory) | High RAM | r7g.2xlarge | Databases, caches, in-memory analytics |
| I / D (storage) | Fast local disks | i4i.xlarge | NoSQL databases, data warehousing |
| P / G / Inf / Trn (accelerated) | GPUs / ML chips | g5.xlarge | Machine learning, video |

**Burstable (T) instances:** earn CPU credits while idle and spend them during bursts. T3/T4g default to **unlimited** mode (extra bursting is charged). Check `CPUCreditBalance` in CloudWatch if a T instance is slow.

**Graviton (ARM)** instances are usually cheaper for the same performance, but your software and container images must support ARM64.

## AMIs (Amazon Machine Images)

| Item | Detail |
|---|---|
| What | Template containing the OS, settings and software for the root volume |
| Sources | AWS (Amazon Linux 2023, Ubuntu, Windows), Marketplace (vendor images, may cost extra), community, **your own** |
| Regional | An AMI ID exists in one region only; copy it to use elsewhere |
| Custom AMI ("golden image") | Install and harden once → create image → launch identical servers fast. Built with tools like Packer or EC2 Image Builder |
| Find latest Amazon Linux | `aws ssm get-parameters --names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 --query 'Parameters[0].Value' --output text` |

**Default login users:** Amazon Linux / RHEL → `ec2-user` · Ubuntu → `ubuntu` · Debian → `admin`.

## Launch settings explained

| Setting | What to choose and why |
|---|---|
| Name & tags | `Name=web-1`, plus `Project`, `Env`, `Owner` |

| Setting | What to choose and why |
|---|---|
| AMI | Amazon Linux 2023 or Ubuntu LTS |
| Instance type | `t3.micro` / `t4g.micro` for labs |
| Key pair | ED25519 key; download the `.pem` once and `chmod 400` it. Or skip keys and use SSM |
| Network | VPC + subnet (public subnet if it must be reachable from the internet) |
| Auto-assign public IP | Enable only for public servers |
| Security group | Allow only needed ports (80/443 for web; 22 from your IP only, or none with SSM) |
| Storage | gp3 root volume, size as needed, **encrypted** |
| IAM instance profile | A role with only the permissions the app needs (and `AmazonSSMManagedInstanceCore` for Session Manager) |
| Metadata options | **IMDSv2 required** (token-based; protects against SSRF credential theft) |
| User data | Bootstrap script (install packages, start services) |
| Termination protection | Enable for important servers |

## Connecting to an instance

| Method | Needs | Notes |
|---|---|---|
| SSH | Key pair, port 22 open to your IP, public IP (or bastion/VPN) | `ssh -i key.pem ubuntu@<public-ip>` |
| EC2 Instance Connect | Supported AMI, port 22 open to the Instance Connect service range | Browser-based SSH, temporary key pushed for 60 s |
| **SSM Session Manager** | SSM agent (preinstalled on Amazon Linux/Ubuntu AMIs), instance role with `AmazonSSMManagedInstanceCore`, outbound access to SSM (internet/NAT or VPC endpoints) | **No open ports, no keys, sessions logged.** Preferred in companies |
| RDP | Windows instances, port 3389 from your IP | Get the password with the key pair |

## User data and instance metadata

**User data** runs **once, as root, at the first boot** (via cloud-init). Logs: `/var/log/cloud-init-output.log`.

```
#!/bin/bash
dnf install -y nginx                        # Amazon Linux 2023 (use apt-get on Ubuntu)
echo "<h1>Hello from $(hostname)</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

**Instance metadata (IMDS)** at `169.254.169.254` lets software on the instance learn about itself (ID, AZ, IPs) and get role credentials. **IMDSv2** requires a session token:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
MD=http://169.254.169.254/latest/meta-data
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/instance-id
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" $MD/placement/availability-zone
```

## Instance lifecycle and billing

```
launch → pending → RUNNING ⇄ stopping → STOPPED → (start) → pending → RUNNING
                          │
                          └──────── terminate ──────► shutting-down → TERMINATED (gone)
```

| State | Compute billed? | EBS billed? | Notes |
|---|---|---|---|
| Running | Yes | Yes | |
| Stopped | No | **Yes** | Public IP is released (unless Elastic IP); instance store data is lost |
| Hibernated | No | Yes | RAM saved to EBS; resumes faster |
| Terminated | No | Root volume deleted by default ( `DeleteOnTermination` ) | Cannot be undone |

**Reboot** keeps the same host, IPs and data. **Stop/start** usually moves the instance to new hardware (fixes some hardware issues).

## Placement groups

| Type | What | Use |
|---|---|---|
| Cluster | Instances close together in one AZ | Lowest latency, HPC |
| Spread | Each instance on different hardware (max 7 per AZ per group) | A few critical instances |
| Partition | Groups of instances on separate racks | Big distributed systems (Kafka, HDFS, Cassandra) |

## EC2 networking

| Item | Detail |
|---|---|
| ENI (Elastic Network Interface) | Virtual network card. Has private IP(s), SGs, MAC. Primary ENI can't be detached |
| Private IP | Stays for the instance's life |
| Public IP | Assigned at launch if enabled; **changes after stop/start** |
| Elastic IP | Static public IPv4 you own until released. Billed hourly like all public IPv4 |
| Source/destination check | Must be **disabled** on NAT instances / appliances that forward traffic |

## Monitoring an instance

| Check | Meaning | Typical fix |
|---|---|---|
| System status check failed | AWS hardware/host problem | Stop and start (moves host) |
| Instance status check failed | Problem inside the OS (boot failure, full disk, bad network config) | Check the **system log / screenshot**, fix OS config, restore from snapshot |
| Attached EBS status check failed | Volume problem | Check volume health, restore from snapshot |

CloudWatch EC2 metrics by default: CPU, network, disk I/O for instance store, status checks — **not memory or disk space** (install the **CloudWatch agent** for those). Basic monitoring = 5-minute data; detailed = 1-minute (paid).

## ⚠️ Lab: launch a web server with the CLI

Cost: `t3.micro` is small but **not free** outside credits; public IPv4 is billed per hour. Delete everything at the end.

```
REGION=ap-south-1
AMI=$(aws ssm get-parameters --region $REGION \
  --names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query 'Parameters[0].Value' --output text)

MYIP=$(curl -s https://checkip.amazonaws.com)/32
VPC=$(aws ec2 describe-vpcs --region $REGION --filters Name=isDefault,Values=true \
  --query 'Vpcs[0].VpcId' --output text)

SG=$(aws ec2 create-security-group --region $REGION --group-name web-lab-sg \
  --description "web lab" --vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --region $REGION --group-id $SG \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --region $REGION --group-id $SG \
  --protocol tcp --port 22 --cidr $MYIP

ID=$(aws ec2 run-instances --region $REGION --image-id $AMI --instance-type t3.micro \
  --security-group-ids $SG --user-data file://userdata.sh \
  --metadata-options HttpTokens=required \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-lab}]' \
  --query 'Instances[0].InstanceId' --output text)

aws ec2 wait instance-running --region $REGION --instance-ids $ID
aws ec2 describe-instances --region $REGION --instance-ids $ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text
# open http://<public-ip> in the browser (give user data ~1 minute)
```

**Cleanup:**

```
aws ec2 terminate-instances --region $REGION --instance-ids $ID
aws ec2 wait instance-terminated --region $REGION --instance-ids $ID
aws ec2 delete-security-group --region $REGION --group-id $SG
```

## EC2 troubleshooting

| Symptom | Check |
|---|---|
| SSH times out | Instance running? Public IP? SG allows 22 from **your current IP**? NACL? Route table has `0.0.0.0/0 → IGW`? Company network blocking 22? |
| `Permission denied (publickey)` | Wrong username (`ec2-user` vs `ubuntu`), wrong key, key file permissions not `400` |
| `UNPROTECTED PRIVATE KEY FILE` | `chmod 400 key.pem` (on Windows: remove other users' access in file properties) |
| Website not loading | SG port 80/443? nginx running (`systemctl status nginx`)? Listening on `0.0.0.0` (`ss -tulpn`)? User data failed (`/var/log/cloud-init-output.log`)? |
| Instance status check failed | Get system log (Actions → Monitor → Get system log), check full disk / fstab errors |

| Symptom | Check |
|---|---|
| Can't launch: `InsufficientInstanceCapacity` | Try another AZ or instance type |
| Can't launch: `VcpuLimitExceeded` | Service Quotas → request an increase |
| Lost key pair | Use SSM Session Manager or EC2 Instance Connect, or attach the root volume to another instance to add a new public key |

## 7.10 Storage for EC2: EBS, instance store, EFS

### EBS (Elastic Block Store)

**What:** network-attached **block** disks for EC2 (like a virtual hard drive). **Key facts:** lives in **one AZ** (attach only to instances in the same AZ) · persists independently of the instance · can be resized online · encrypted with KMS · backed up with **snapshots**.

| Volume type | Kind | Performance model | Use |
|---|---|---|---|
| **gp3** | SSD | Baseline 3,000 IOPS and 125 MB/s at any size; more IOPS/throughput can be provisioned separately | **Default choice**: boot volumes, most apps |
| gp2 | SSD (older) | IOPS grow with size (3 IOPS/GB, burst) | Legacy; migrate to gp3 (cheaper) |
| io2 / io2 Block Express | SSD | Provisioned IOPS, highest durability | Critical databases |
| st1 | HDD | Throughput-optimized | Big sequential data, logs |
| sc1 | HDD | Cheapest, cold | Rarely accessed data |

- **IOPS** = input/output operations per second (many small reads/writes, e.g. databases). **Throughput** = MB/s (large sequential reads/writes).
- **Multi-Attach** (io1/io2 only): attach one volume to several instances in the same AZ (needs a cluster-aware file system).
- **Encryption:** turn on **EBS encryption by default** per region (EC2 settings). Snapshots of encrypted volumes are encrypted.

**Snapshots:** - Point-in-time backups stored in S3 (managed by AWS), **incremental** (only changed blocks are stored after the first). - Restore = create a new volume from the snapshot (in **any AZ** of the region), or copy the snapshot to another region (DR). - Automate with **Data Lifecycle Manager** or **AWS Backup**. - Old snapshots cost money: set retention.

### ⚠️ Lab: add, format, mount and grow a volume

```
# 1) Create a 10 GiB gp3 volume in the SAME AZ as the instance, then attach it
aws ec2 create-volume --availability-zone ap-south-1a --size 10 --volume-type gp3 --encrypted \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=data-lab}]'
aws ec2 attach-volume --volume-id vol-0123456789abcdef0 \
  --instance-id i-0123456789abcdef0 --device /dev/sdf

# 2) On the instance
lsblk                                  # new disk appears, e.g. nvme1n1 (Nitro names it nvme*)
sudo mkfs -t xfs /dev/nvme1n1          # format (ONLY on a new, empty disk!)
sudo mkdir /data && sudo mount /dev/nvme1n1 /data
df -h /data

# 3) Mount automatically at boot, using the UUID (device names can change)
sudo blkid /dev/nvme1n1                # copy the UUID
echo 'UUID=<uuid> /data xfs defaults,nofail 0 2' | sudo tee -a /etc/fstab
sudo umount /data && sudo mount -a && df -h /data   # test fstab before rebooting

# 4) Grow the volume later (no downtime)
aws ec2 modify-volume --volume-id vol-0123456789abcdef0 --size 20
sudo xfs_growfs /data                  # ext4 would be: sudo resize2fs /dev/nvme1n1
```

**Snapshot:** `aws ec2 create-snapshot --volume-id vol-0123456789abcdef0 --description "data-lab backup"` . **Cleanup:** unmount, remove the fstab line, detach, delete the volume and snapshot.

`nofail` in fstab prevents the instance from failing to boot if the volume is missing.

### Instance store

Physical disks on the host machine (instance types with `d`, e.g. `m6id`). Very fast, **but data is lost on stop, hibernate or terminate** (it survives a reboot). Use only for caches, buffers, scratch data, or replicated data.

### EFS (Elastic File System)

**What:** managed **NFS** file system shared by many instances (and containers/Lambda), across **multiple AZs**, grows and shrinks automatically.

| Setting | Options |
|---|---|
| Storage classes | Standard, Infrequent Access, Archive (lifecycle policies move files automatically) |
| Availability | Regional (multi-AZ) or One Zone (cheaper) |
| Throughput mode | Elastic (recommended), Provisioned, Bursting |
| Access | Mount targets in each AZ subnet; SG must allow **NFS port 2049** from the clients |

Mount: `sudo mount -t efs fs-0123456789abcdef0:/ /mnt/efs` (with `amazon-efs-utils`) or standard NFS mount. Linux only.

| | EBS | Instance store | EFS | S3 |
|---|---|---|---|---|
| Type | Block | Block | File (NFS) | Object |
| Shared by many instances | No (except Multi-Attach) | No | **Yes** | Yes (via API) |
| Survives stop | Yes | **No** | Yes | Yes |
| Scope | One AZ | One host | Region (multi-AZ) | Region |

| | EBS | Instance store | EFS | S3 |
|---|---|---|---|---|
| Typical use | OS, databases | Cache | Shared content, home dirs | Files, backups, static sites, data lakes |

**FSx** (awareness): managed Windows File Server, Lustre (HPC), NetApp ONTAP, OpenZFS.

## 7.11 S3: Simple Storage Service

**What: object storage**: store any number of files ("objects") in containers ("buckets") and access them over HTTPS/API. **Why:** practically unlimited, very durable (designed for 99.999999999% = "11 nines"), cheap, integrated with almost every AWS service. **In companies:** static websites and assets, backups, logs, data lakes, Terraform state, CI artifacts, file uploads from apps.

### Core concepts

| Term | Meaning |
|---|---|
| Bucket | Container for objects. Name is **globally unique**, 3–63 characters, lowercase letters, numbers, hyphens, dots. Created in one region |
| Object | A file + metadata. Up to 5 TB each |
| Key | The object's full name: `logs/2026/10/app.log`. "Folders" are just key **prefixes** |
| Version ID | Present when versioning is on |
| Consistency | **Strong read-after-write** consistency for all operations |

**Uploads:** single PUT up to 5 GB; use **multipart upload** for large files (recommended above ~100 MB; the CLI does it automatically).

### Storage classes

| Class | Availability zones | Min. storage duration | Retrieval | Use |
|---|---|---|---|---|
| Standard | 3+ AZs | — | Instant | Frequently accessed data |
| Intelligent-Tiering | 3+ AZs | — | Instant (archive tiers optional) | Unknown or changing access patterns |
| Standard-IA | 3+ AZs | 30 days | Instant, per-GB retrieval fee | Infrequent but needs fast access (backups) |
| One Zone-IA | **1 AZ** | 30 days | Instant, retrieval fee | Re-creatable infrequent data |
| Glacier Instant Retrieval | 3+ AZs | 90 days | Milliseconds | Archives accessed ~quarterly |
| Glacier Flexible Retrieval | 3+ AZs | 90 days | Minutes to hours | Archives, DR copies |
| Glacier Deep Archive | 3+ AZs | 180 days | Within ~12–48 hours | Long-term compliance archives (cheapest) |

### Lifecycle rules

Automatically **transition** objects to cheaper classes or **expire** (delete) them after N days.

```
{
  "Rules": [{
    "ID": "logs-retention",
    "Filter": { "Prefix": "logs/" },
    "Status": "Enabled",
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 365 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 }
  }]
}
```

Apply: `aws s3api put-bucket-lifecycle-configuration --bucket <bucket> --lifecycle-configuration file://lifecycle.json`.

## Versioning, Object Lock, replication

| Feature | What it does | Use |
|---|---|---|
| Versioning | Keeps every version; a delete adds a "delete marker" instead of removing data | Protect against accidental overwrite/delete. **Turn on for important buckets** (and Terraform state) |
| MFA Delete | Requires MFA to permanently delete versions or change versioning | Extra protection (root only) |
| Object Lock | Write-once-read-many (WORM): objects can't be deleted until a retention date | Compliance, ransomware protection |
| Replication (CRR / SRR) | Copy new objects automatically to another bucket in another (CRR) or same (SRR) region | DR, compliance, log aggregation. Requires versioning on both buckets |

## Security and access control

**Layers that decide access:**

```
Block Public Access (account + bucket)  ── overrides anything that would make data public
        │
IAM policies (identity-based)            ── what users/roles can do
Bucket policy (resource-based)           ── who can access this bucket (other accounts, services)
ACLs                                     ── legacy; disabled by default (bucket owner enforced)
        │
Encryption (SSE-S3 default, SSE-KMS)     ── KMS key policy must also allow decrypt for SSE-KMS
```

| Control | Detail |
|---|---|
| Block Public Access | **On by default** for new buckets. Leave it on unless the bucket truly must be public |
| Bucket policy | JSON with `Principal`. E.g. allow only CloudFront, enforce HTTPS, enforce encryption |
| Default encryption | All new objects are encrypted (SSE-S3 by default). Use **SSE-KMS** for audit and key control; **DSSE-KMS** for dual-layer; SSE-C = your own key per request |
| Presigned URL | Temporary link to one private object: `aws s3 presign s3://<bucket>/report.pdf --expires-in 3600` |
| Access points | Named entry points with their own policy for large shared buckets |

| Control | Detail |
|---|---|
| VPC gateway endpoint | Private S3 access from a VPC without internet/NAT |

**Bucket policy: deny any request that is not HTTPS:**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyInsecureTransport",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": ["arn:aws:s3:::<bucket>", "arn:aws:s3:::<bucket>/*"],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  }]
}
```

## Static website hosting

| Approach | How | When |
|---|---|---|
| S3 website endpoint | Enable static website hosting, make objects public (Block Public Access off for that bucket + public-read bucket policy) | Quick labs only. HTTP only |
| **S3 + CloudFront (recommended)** | Bucket stays **private**; CloudFront with **Origin Access Control (OAC)** reads it; HTTPS with ACM certificate; custom domain via Route 53 | Production static sites |

## Other features

| Feature | Use |
|---|---|
| Event notifications | Trigger Lambda/SQS/SNS/EventBridge on upload/delete (e.g. make thumbnails) |
| Transfer Acceleration | Faster long-distance uploads via edge locations |
| Server access logs / CloudTrail data events | Who accessed what |
| S3 Inventory / Storage Lens | Reports and analytics about your storage |
| Requester Pays | The downloader pays transfer costs |

**Pricing factors:** storage (GB-month by class) + requests (PUT/GET) + data transfer out + retrieval fees (IA/Glacier) + features (replication, Intelligent-Tiering monitoring).

## S3 CLI

| Command | Purpose |
|---|---|
| `aws s3 mb s3://<bucket> --region ap-south-1` | Make bucket |
| `aws s3 ls` / `aws s3 ls s3://<bucket>/logs/` | List buckets / objects under a prefix |
| `aws s3 cp file.txt s3://<bucket>/docs/` | Upload ( `cp s3://... .` to download) |
| `aws s3 cp dir s3://<bucket>/dir --recursive` | Upload a folder |

| Command | Purpose |
|---|---|
| `aws s3 sync ./site s3://<bucket> --delete` | Make the bucket match a local folder |
| `aws s3 mv`, `aws s3 rm s3://<bucket>/file` | Move / delete |
| `aws s3 rb s3://<bucket> --force` | Delete bucket and all objects (not old versions) |
| `aws s3api put-bucket-versioning --bucket <bucket> --versioning-configuration Status=Enabled` | Turn on versioning |
| `aws s3api get-bucket-policy --bucket <bucket>` | Show bucket policy |
| `aws s3 presign s3://<bucket>/file --expires-in 600` | Temporary download link |

`aws s3` = high-level, file-like commands. `aws s3api` = every S3 API operation (settings, policies, versions).

### ⚠️ Lab: private bucket, versioning, lifecycle (almost free)

1. `aws s3 mb s3://<yourname>-lab-<random>` (Block Public Access stays on).
2. Enable versioning (command above). Upload `notes.txt` twice with different content.
3. `aws s3api list-object-versions --bucket <bucket>` → see two versions. Delete the object and list again → see the **delete marker**. Restore by deleting the delete marker version.
4. Apply the lifecycle JSON above.
5. Create a presigned URL and open it in a browser; open the plain object URL → **AccessDenied** (private).
6. **Cleanup:** delete all versions (console: Empty bucket) then `aws s3 rb s3://<bucket>`.

### S3 troubleshooting

| Symptom | Likely cause |
|---|---|
| `AccessDenied` reading an object | IAM missing `s3:GetObject` on `bucket/*`; bucket policy Deny; object encrypted with KMS and no `kms:Decrypt`; object owned by another account (old ACL setups) |
| `AccessDenied` listing | Missing `s3:ListBucket` on the **bucket** ARN (not `/*`) |
| Website 403 | Block Public Access on / no public bucket policy (or use CloudFront OAC) / missing `index.html` |
| `BucketAlreadyExists` | Name taken by someone else globally: choose a unique name |
| Bucket won't delete | Not empty, or old versions/delete markers remain |
| Unexpected costs | Old versions without lifecycle rules, Glacier minimum duration fees, data transfer out |

## 7.12 VPC: Virtual Private Cloud

**What:** your own private, isolated network inside an AWS region. You choose the IP range, subnets, routing and firewalls. **Why:** every EC2 instance, RDS database, load balancer, Lambda-in-VPC and EKS cluster lives in a VPC. Network design decides security, availability and cost. **In companies:** each environment (dev/staging/prod) usually has its own VPC, often in its own account, connected through Transit Gateway.

## Reference design (2 AZs, public + private + database tiers)

```
Region ap-south-1 — VPC 10.0.0.0/16
│
├─ Internet Gateway (IGW) ◄──────────────── internet
│
├─ AZ a                                    AZ b
│  ├─ Public  10.0.1.0/24                 ├─ Public  10.0.2.0/24
│  │   ALB node, NAT Gateway a             │   ALB node, NAT Gateway b
│  │   route: 0.0.0.0/0 → IGW               │   route: 0.0.0.0/0 → IGW
│  ├─ Private 10.0.11.0/24 (app)           ├─ Private 10.0.12.0/24 (app)
│  │   EC2 / ECS / EKS nodes                │   EC2 / ECS / EKS nodes
│  │   route: 0.0.0.0/0 → NAT a             │   route: 0.0.0.0/0 → NAT b
│  └─ Private 10.0.21.0/24 (data)           └─ Private 10.0.22.0/24 (data)
│      RDS primary                             RDS standby
│      no internet route                       no internet route
│
└─ Gateway endpoint for S3 (private S3 access, free)
```

## CIDR planning

| Rule | Detail |
|---|---|
| VPC size | Between `/16` (65,536 IPs) and `/28` (16 IPs). `/16` is common |
| Private ranges | Use RFC 1918: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` |
| **No overlaps** | Never overlap with other VPCs, on-premises networks or VPNs you may connect later. Peering/TGW/VPN can't route overlapping ranges |
| Secondary CIDRs | Extra ranges can be added to a VPC later |
| Reserved IPs | AWS reserves **5 IPs per subnet**: `.0` network, `.1` VPC router, `.2` DNS, `.3` future use, last = broadcast. A `/24` gives **251** usable |
| Leave room | Allocate bigger blocks than you need now; EKS consumes many IPs (one per pod with the VPC CNI) |

**Default VPC:** every region has one (`172.31.0.0/16`, a public `/20` subnet per AZ, IGW attached). Fine for quick labs; build your own VPCs for real work.

## Subnets

- A subnet is a slice of the VPC CIDR in **exactly one AZ**.
- **Public subnet** = its route table has `0.0.0.0/0 → Internet Gateway`. Instances also need a public IP to be reachable.
- **Private subnet** = no route to an IGW. Outbound internet (updates, APIs) goes through a **NAT Gateway**.
- **Isolated subnet** = no internet route at all (databases). Use VPC endpoints for AWS services.
- Setting `map-public-ip-on-launch` gives instances a public IP automatically (public subnets only).

## Route tables

| Concept | Detail |
|---|---|
| Route table | Rules: destination CIDR → target |
| `local` route | Automatic route for the VPC CIDR; all subnets can reach each other (SGs/NACLs still apply) |

| Concept | Detail |
|---|---|
| Main route table | Used by subnets without an explicit association. Keep it private; create separate public tables |
| Association | Each subnet uses exactly one route table |
| Longest prefix wins | `10.0.5.0/24` beats `10.0.0.0/16` beats `0.0.0.0/0` |
| Targets | `igw-`, `nat-`, `pcx-` (peering), `tgw-`, `vpce-` (gateway endpoint), `eni-`, virtual private gateway ( `vgw-` ) |

## Internet Gateway and NAT Gateway

| | Internet Gateway | NAT Gateway |
|---|---|---|
| Purpose | Two-way internet access for resources with public IPs | **Outbound-only** internet for private subnets |
| Placement | Attached to the VPC (one per VPC) | Inside a **public** subnet, with an Elastic IP |
| Cost | Free (data transfer charged) | **Hourly + per GB processed**: one of the most common surprise bills |
| HA | Built-in | Zonal: create **one per AZ** for HA, each private subnet routes to the NAT in its own AZ |

A **NAT instance** (EC2 doing NAT, source/dest check disabled) is cheaper but you manage HA, patching and scaling: only for labs or very low budgets. For IPv6, use an **egress-only internet gateway** for outbound-only access.

## Security Groups (SG) in depth

| Property | Detail |
|---|---|
| Level | Attached to network interfaces (instances, ALBs, RDS, Lambda ENIs…) |
| Stateful | Return traffic is automatically allowed |
| Rules | **Allow only.** Default: no inbound, all outbound |
| Sources | CIDR, prefix list, or **another security group** (best practice for tiers) |
| Evaluation | All rules are combined; no order |

**Tiered SG design (SG references instead of IPs):**

| SG | Inbound rule |
|---|---|
| `alb-sg` | 443 (and 80 for redirect) from `0.0.0.0/0` |
| `app-sg` | 8080 from **`alb-sg`** |
| `db-sg` | 5432 from **`app-sg`** |
| `bastion` | Not needed: use SSM Session Manager |

## Network ACLs (NACL) in depth

| Property | Detail |
|---|---|
| Level | Subnet boundary |

| Property | Detail |
|---|---|
| Stateless | Return traffic needs its own rule |
| Rules | Allow **and** Deny, numbered; **lowest number first, first match wins**; final `*` rule denies |
| Default NACL | Allows everything in and out |
| Custom NACL | Denies everything until you add rules |
| Ephemeral ports | Return traffic to clients uses ports **1024–65535**, so outbound (for servers) and inbound (for clients) rules must allow them |

**Example NACL for a public web subnet:**

| Direction | Rule # | Protocol | Port | Source/Dest | Action |
|---|---|---|---|---|---|
| Inbound | 100 | TCP | 443 | 0.0.0.0/0 | Allow |
| Inbound | 110 | TCP | 1024–65535 | 0.0.0.0/0 | Allow (responses to outbound calls) |
| Inbound | 120 | TCP | 22 | 198.51.100.7/32 | Deny (block a bad actor before broad allows) |
| Outbound | 100 | TCP | 1024–65535 | 0.0.0.0/0 | Allow (responses to clients) |
| Outbound | 110 | TCP | 443 | 0.0.0.0/0 | Allow (calls to the internet) |

**Use NACLs** for coarse subnet-wide blocks (deny an IP range); do day-to-day filtering with SGs.

## VPC endpoints (private access to AWS services)

| Type | Services | How | Cost |
|---|---|---|---|
| **Gateway endpoint** | **S3 and DynamoDB only** | Adds a route in the route table | Free |
| **Interface endpoint (PrivateLink)** | Most services (ECR, SSM, Secrets Manager, STS, CloudWatch, KMS…) and your own services | Creates ENIs with private IPs in your subnets; private DNS makes the normal service name resolve to them | Hourly per AZ + per GB |

**Why:** keep traffic off the internet, reduce NAT Gateway costs, allow fully isolated subnets. Endpoint policies can restrict what's reachable (e.g. only your buckets).

## Connecting VPCs and networks

| Option | What | Notes |
|---|---|---|
| VPC peering | Private connection between two VPCs (same/different account or region) | **Not transitive** (A↔B and B↔C does not give A↔C); CIDRs must not overlap; update route tables on both sides |
| Transit Gateway (TGW) | Regional hub connecting many VPCs, VPNs and Direct Connect | Hub-and-spoke, transitive routing, route tables for segmentation. Standard for multi-account networks |
| PrivateLink | Expose one service privately to other VPCs/accounts via an NLB | Consumer creates an interface endpoint; no routing between whole networks |

| Option | What | Notes |
|---|---|---|
| Site-to-Site VPN | IPsec tunnels over the internet to on-premises (2 tunnels per connection) | Quick to set up; terminates on a virtual private gateway or TGW; static or BGP routing |
| Direct Connect (DX) | Dedicated private fibre link to AWS | Consistent latency, high bandwidth, weeks to provision; combine with VPN for encryption/backup |
| Client VPN | Managed OpenVPN for users | Remote access to VPCs |
| Cloud WAN | Managed global network across regions | Large enterprises |

## DNS inside a VPC

- The **Route 53 Resolver** at VPC base + 2 (e.g. `10.0.0.2`) answers DNS for instances.
- VPC settings: `enableDnsSupport` (use the resolver) and `enableDnsHostnames` (give instances DNS names). Both must be on for private hosted zones and interface endpoint private DNS.
- **Private hosted zones**: internal names like `db.internal.shop` visible only in associated VPCs.
- **Resolver endpoints** (inbound/outbound) + forwarding rules connect DNS between AWS and on-premises.

## IPv6 in a VPC

Add an Amazon-provided IPv6 `/56` to the VPC and `/64` to subnets. All IPv6 addresses are public, so use **security groups** and an **egress-only IGW** for outbound-only access. Dual-stack (IPv4 + IPv6) is common.

## Visibility and troubleshooting tools

| Tool | Use |
|---|---|
| **VPC Flow Logs** | Records accepted/rejected traffic per ENI/subnet/VPC to CloudWatch Logs or S3. Fields include srcaddr, dstaddr, srcport, dstport, protocol, action (ACCEPT/REJECT) |
| **Reachability Analyzer** | Tells you whether A can reach B and which component blocks it (SG, NACL, route) |
| Network Access Analyzer | Finds unintended network paths (e.g. databases reachable from the internet) |
| Traffic Mirroring | Copy packets to an analysis appliance (IDS, Wireshark) |

**Flow log line example:** `2 123456789012 eni-0a1b2c3d 10.0.1.25 10.0.11.40 49152 8080 6 10 840 1700000000 1700000060 REJECT OK` → TCP (6) from 10.0.1.25 to port 8080 on 10.0.11.40 was **rejected** → check `app-sg` allows 8080 from the ALB.

### ⚠️ Lab: build a VPC with the CLI (free except NAT if you add it)

```
R=ap-south-1
VPC=$(aws ec2 create-vpc --region $R --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=lab-vpc}]' \
  --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --region $R --vpc-id $VPC --enable-dns-hostnames '{"Value":true}'

PUB=$(aws ec2 create-subnet --region $R --vpc-id $VPC --cidr-block 10.0.1.0/24 \
  --availability-zone ${R}a --query Subnet.SubnetId --output text)
PRIV=$(aws ec2 create-subnet --region $R --vpc-id $VPC --cidr-block 10.0.11.0/24 \
  --availability-zone ${R}a --query Subnet.SubnetId --output text)
aws ec2 modify-subnet-attribute --region $R --subnet-id $PUB --map-public-ip-on-launch

IGW=$(aws ec2 create-internet-gateway --region $R \
  --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --region $R --internet-gateway-id $IGW --vpc-id $VPC

RT=$(aws ec2 create-route-table --region $R --vpc-id $VPC \
  --query RouteTable.RouteTableId --output text)
aws ec2 create-route --region $R --route-table-id $RT \
  --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
aws ec2 associate-route-table --region $R --route-table-id $RT --subnet-id $PUB
```

**Test:** launch one instance in `$PUB` (gets a public IP, reachable) and one in `$PRIV` (no public IP; reach it from the public one by private IP). From the private instance, `curl https://example.com` fails → no NAT. Optionally add a NAT Gateway (⚠️ hourly cost) in `$PUB`, route `0.0.0.0/0` from the private route table to it, test, and delete it.

**Cleanup (reverse order):** terminate instances → delete NAT Gateway and release its EIP → disassociate/delete the route table → detach and delete the IGW → delete subnets → delete the VPC.

### VPC troubleshooting

| Symptom | Check in order |
|---|---|
| Instance not reachable from internet | Public subnet route to IGW · public IP/EIP · SG inbound · NACL inbound **and** outbound ephemeral · OS firewall · service listening |
| Private instance can't reach internet | Route `0.0.0.0/0 → nat-` in **its** route table · NAT in a **public** subnet · NAT subnet routes to IGW · SG/NACL outbound · NAT state "available" |
| Peered VPCs can't talk | Routes on **both** sides · SGs allow the peer CIDR · no overlapping CIDRs · DNS resolution option for peering |
| Can't reach S3 privately | Gateway endpoint route on that subnet's route table · endpoint policy · bucket policy conditions ( `aws:sourceVpce` ) |
| Interface endpoint not used | Private DNS enabled? VPC DNS settings on? Endpoint SG allows 443 from clients? |
| Out of IPs | Subnet too small (remember 5 reserved, EKS uses many) → add a secondary CIDR / bigger subnets |

## 7.13 Elastic Load Balancing (ELB)

**What:** managed load balancers that spread traffic across many targets and send traffic only to **healthy** ones. **Why:** high availability (multi-AZ), scaling, TLS termination, a single stable entry point.

## Types

| Type | Layer | Protocols | Key features | Use |
|---|---|---|---|---|
| **ALB** (Application) | 7 | HTTP, HTTPS, gRPC, WebSocket | Path/host/header/query routing, redirects, fixed responses, auth (Cognito/OIDC), Lambda targets, WAF | Web apps, APIs, microservices, Kubernetes Ingress |
| **NLB** (Network) | 4 | TCP, UDP, TLS | Millions of requests/s, very low latency, **static IP per AZ** / Elastic IPs, preserves client IP, PrivateLink | Non-HTTP protocols, extreme performance, fixed IPs |
| GWLB (Gateway) | 3 | IP (GENEVE 6081) | Inserts firewall/IDS appliances transparently | Security appliances |
| CLB (Classic) | 4/7 | Legacy | — | Migrate away |

## Components

```
Client ──► DNS name (my-alb-123.ap-south-1.elb.amazonaws.com)
              │
         LISTENER  (HTTPS :443, certificate from ACM)
            │ RULES (evaluated by priority)
            │   host = api.shop.com      → TG-api
            │   path = /images/*         → TG-images
            │   default                  → TG-web
            ▼
       TARGET GROUP (protocol/port, health check) ──► targets: instances | IPs | Lambda
```

| Component | Detail |
|---|---|
| Listener | Port + protocol (+ TLS certificate & security policy for HTTPS) |
| Rules (ALB) | Conditions (host, path, header, method, query, source IP) → actions (forward, redirect, fixed-response, authenticate) |
| Target group | Group of targets + health check. Target types: **instance**, **ip** (containers, on-prem), **lambda**, **alb** (for NLB) |
| Health check | Protocol, path (e.g. `/health`), port, interval (default 30 s), timeout, healthy/unhealthy thresholds, success codes |
| Deregistration delay | Time to finish in-flight requests before removing a target (default 300 s; often lowered to 30–60 s) |
| Stickiness | Sends a user to the same target (cookie). Avoid if the app can be stateless |
| Cross-zone load balancing | Spread evenly across targets in all AZs (on by default for ALB) |
| Access logs | Per-request logs to S3 |

**Client IP behind an ALB:** the app sees the ALB's IP; the real client IP is in the `X-Forwarded-For` header (and `X-Forwarded-Proto` tells HTTP vs HTTPS).

**HTTPS best practice:** listener 443 with an **ACM** certificate, listener 80 with a **redirect to 443**, modern TLS security policy, SG of targets allows traffic only from the ALB SG.

### ALB error codes

| Code | Meaning | Usual cause |
|---|---|---|
| 502 Bad Gateway | Target sent an invalid response or closed the connection | App crashed, wrong port, keep-alive timeout shorter than ALB idle timeout |
| 503 Service Unavailable | **No healthy targets** in the target group | All targets failing health checks, or none registered |
| 504 Gateway Timeout | Target didn't answer in time | Slow app/database, idle timeout (default 60 s) too short, SG/NACL dropping traffic |
| 460 / 463 | Client closed connection / too many IPs in X-Forwarded-For | Client side |

### ⚠️ Lab: ALB in front of two instances (hourly cost: delete after)

1. Launch two web instances (user data from 7.9) in **two different AZs**, SG `app-sg` allowing 80 from `alb-sg`.
2. Create target group `web-tg` (HTTP 80, health check `/`) and register both instances.
3. Create an internet-facing ALB in two public subnets with `alb-sg` (80 from anywhere), listener 80 → `web-tg`.
4. Open the ALB DNS name and refresh: the hostname alternates between the two instances.
5. Stop nginx on one instance → after the unhealthy threshold, all traffic goes to the other. Start it again.
6. **Cleanup:** delete ALB → target group → instances → security groups.

## 7.14 EC2 Auto Scaling

**What:** keeps the right number of instances running: replaces unhealthy ones and adds/removes instances based on demand. **Why:** self-healing, high availability, cost saving (scale in when quiet).

| Component | Detail |
|---|---|
| **Launch template** | Versioned recipe: AMI, instance type, key, SGs, IAM profile, user data, metadata options, tags |
| **Auto Scaling Group (ASG)** | Min / desired / max capacity, subnets (spread across AZs), target groups, health check type |
| Health check type | `EC2` (status checks only) or `ELB` (also the load balancer health check: recommended with an ALB) |
| Health check grace period | Seconds to wait after launch before health checks count (time for user data to finish) |
| Instance warmup / cooldown | Prevents scaling again before new instances are useful |
| Termination policy | Which instance to remove first (default: balance AZs, then oldest launch template…) |
| Lifecycle hooks | Pause instances on launch/terminate to run scripts (e.g. drain, copy logs) |
| Instance refresh | Rolling replacement of all instances with a new launch template version (deployments) |
| Warm pools | Pre-initialized stopped instances for faster scale-out |
| Mixed instances policy | Several instance types + On-Demand/Spot mix |

### Scaling policies

| Policy | How | Example |
|---|---|---|
| **Target tracking** (recommended) | Keep a metric at a target value | Average CPU = 50%, or ALB requests per target = 1,000 |
| Step scaling | Different adjustments by alarm breach size | +1 at 70% CPU, +3 at 90% |
| Simple scaling | One adjustment per alarm + cooldown | Legacy |
| Scheduled | At specific times | Scale to 10 at 08:00 weekdays, back to 2 at 20:00 |
| Predictive | ML forecast from history | Daily traffic patterns |

```
aws autoscaling put-scaling-policy --auto-scaling-group-name web-asg \
  --policy-name cpu50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration file://cpu50.json

# cpu50.json
# {
#   "PredefinedMetricSpecification": { "PredefinedMetricType": "ASGAverageCPUUtilization" },
#   "TargetValue": 50.0
# }
```

**Golden rule:** instances in an ASG must be **disposable**: no data stored only on the instance (use RDS, S3, EFS), configuration from user data / AMI, logs shipped to CloudWatch.

### ASG troubleshooting

| Symptom | Cause |
|---|---|
| Instances launch and terminate in a loop | Failing ELB health checks (app not ready, wrong health check path/port, grace period too short) |
| Not scaling out | Max capacity reached, metric not breaching, cooldown, vCPU quota, no capacity for the type |
| Launch failures | Check **Activity history**: invalid AMI, missing IAM permission for the instance profile, subnet out of IPs |

## 7.15 Route 53 (DNS)

**What:** AWS's highly available DNS service, domain registrar, and health checker (port 53 → "Route 53").

| Concept | Detail |
|---|---|
| Hosted zone | Records for a domain. **Public** (internet) or **private** (inside associated VPCs) |
| NS / SOA | Created automatically; set the NS records at your registrar if the domain is registered elsewhere |
| Record types | A, AAAA, CNAME, MX, TXT, NS, SOA, PTR, SRV, CAA |
| **Alias record** | AWS extension: points to AWS resources (ALB, CloudFront, S3 website, API Gateway, another record). Works at the **zone apex** ( `shop.com` ), free queries, no TTL to set |
| CNAME | Alias name → another name. **Not allowed at the zone apex** |

| Concept | Detail |
|---|---|
| TTL | Seconds resolvers cache the answer. Lower it (e.g. 60) before migrations |

### Routing policies

| Policy | Behaviour | Use |
|---|---|---|
| Simple | One record, one or more values | Single resource |
| Weighted | Split by weights (e.g. 90/10) | Canary releases, A/B tests |
| Latency | Lowest-latency region for the user | Multi-region apps |
| Failover | Primary/secondary with health checks | Active-passive DR |
| Geolocation | By user's country/continent | Localized content, legal restrictions |
| Geoproximity | By distance with a bias | Shift traffic between regions |
| Multivalue answer | Up to 8 healthy random records | Simple client-side load spreading |
| IP-based | By client IP ranges (CIDR) | ISP-specific routing |

**Health checks** monitor an endpoint (HTTP/HTTPS/TCP), another health check, or a CloudWatch alarm; failover policies use them.

```
aws route53 list-hosted-zones
aws route53 list-resource-record-sets --hosted-zone-id Z0123456789ABC
dig +short app.shop.com            # verify from your machine
```

## 7.16 CloudFront and Global Accelerator

### CloudFront (CDN)

**What:** content delivery network caching content at hundreds of edge locations. **Why:** faster for users worldwide, less load on origins, HTTPS everywhere, DDoS protection (Shield Standard), WAF integration.

| Concept | Detail |
|---|---|
| Distribution | The CDN configuration with a domain like `d111111abcdef8.cloudfront.net` (+ your own domain via ACM certificate in **us-east-1**) |
| Origin | Where content comes from: S3 bucket, ALB, EC2, API Gateway, any HTTP server |
| Origin Access Control (OAC) | Lets CloudFront read a **private** S3 bucket (bucket policy allows only that distribution) |
| Behaviors | Path patterns ( `/api/*` , `/static/*` ) → origin + cache settings |
| Cache policy / TTL | What's part of the cache key (headers, cookies, query strings) and how long to cache |
| Invalidation | Remove cached files early: `aws cloudfront create-invalidation --distribution-id E123 --paths "/index.html"` (prefer versioned file names) |
| Signed URLs / cookies | Private content for paying/logged-in users |
| Edge functions | CloudFront Functions (lightweight header/URL logic) and Lambda@Edge (heavier logic) |

### Global Accelerator

Two static **anycast** IPs that route users over the AWS global network to the best healthy regional endpoint (ALB, NLB, EC2). For non-HTTP traffic, fixed IPs, and fast regional failover. CloudFront = caching for HTTP; Global Accelerator = network path optimization, no caching.

## 7.17 Databases on AWS

### Choosing a database

| Need | Service | Type |
|---|---|---|
| Relational (SQL), standard engines | **RDS** (MySQL, PostgreSQL, MariaDB, SQL Server, Oracle, Db2) | Managed relational |
| Relational, highest performance/availability | **Aurora** (MySQL/PostgreSQL compatible) | Cloud-native relational |
| Key-value / document at any scale, serverless | **DynamoDB** | NoSQL |
| In-memory cache, sessions, leaderboards | **ElastiCache** (Redis OSS / Valkey / Memcached), **MemoryDB** (durable Redis) | In-memory |
| MongoDB-compatible documents | DocumentDB | Document |
| Graph (relationships, fraud, social) | Neptune | Graph |
| Cassandra-compatible wide-column | Keyspaces | Wide-column |
| Time series (IoT, metrics) | Timestream | Time series |
| Search and log analytics | OpenSearch Service | Search engine |
| Data warehouse (analytics on huge data) | Redshift | Columnar warehouse |

**Managed vs self-managed:** you *can* install a database on EC2, but managed services handle patching, backups, failover, replication and monitoring. Choose self-managed only for special requirements.

### RDS (Relational Database Service)

| Topic | Detail |
|---|---|
| Instance class | `db.t4g.micro` (labs), `db.m7g` (general), `db.r7g` (memory-heavy) |
| Storage | gp3 / io2, **storage autoscaling** up to a max you set |
| Network | **DB subnet group** (private subnets in 2+ AZs), SG allowing the DB port only from the app SG, **Publicly accessible = No** |
| **Multi-AZ (instance)** | Synchronous **standby** in another AZ, **automatic failover** (typically 1–2 minutes); the standby is **not readable**. For **availability** |
| Multi-AZ DB cluster | Writer + two **readable** standbys in 3 AZs, faster failover (MySQL/PostgreSQL) |
| **Read replicas** | Asynchronous copies (same or other region) for **read scaling** and DR; can be promoted to standalone |
| Automated backups | Daily snapshot + transaction logs → **point-in-time restore (PITR)** to any second in the retention window (up to 35 days) |

| Topic | Detail |
|---|---|
| Manual snapshots | Kept until you delete them; copy across regions/accounts |
| Restore | Always creates a **new** DB instance (new endpoint) |
| Parameter group | Engine settings (e.g. `max_connections`) |
| Option group | Engine features (Oracle/SQL Server options) |
| Maintenance window | When AWS applies patches/upgrades |
| Encryption | At rest with KMS (choose at creation; to encrypt an existing DB: snapshot → copy encrypted → restore); in transit with TLS |
| Authentication | Password (store in **Secrets Manager**, with rotation), or **IAM database authentication** |
| RDS Proxy | Connection pooling: protects the DB from too many connections (Lambda, spiky apps); faster failover |
| Monitoring | CloudWatch metrics, **Performance Insights / Database Insights** (top SQL, waits), Enhanced Monitoring (OS level), engine logs to CloudWatch |

**Endpoint usage:** the app connects to the **DB endpoint DNS name** (e.g. `shop-db.abc123.ap-south-1.rds.amazonaws.com:5432`), never an IP: the name follows failover.

| Multi-AZ vs read replica | Multi-AZ | Read replica |
|---|---|---|
| Purpose | High availability | Read scaling (and DR) |
| Replication | Synchronous | Asynchronous (replica lag) |
| Readable | No (instance deployment) | Yes |
| Failover | Automatic | Manual promotion |

## Aurora

- Storage is a shared cluster volume with **6 copies across 3 AZs**, grows automatically.
- Up to **15 Aurora Replicas** sharing the same storage (low lag); automatic failover to a replica (usually under ~30 s).
- Endpoints: **cluster (writer)** endpoint, **reader** endpoint (load-balances replicas), custom endpoints.
- **Aurora Serverless v2:** capacity scales in fine-grained steps automatically (ACUs).
- **Global Database:** cross-region replication with fast cross-region recovery.
- Backtrack (MySQL), fast cloning.

## DynamoDB

| Concept | Detail |
|---|---|
| Table / item / attribute | Item = row (max 400 KB), schemaless except the key |
| **Partition key** | Required; decides which partition stores the item. Choose a key with **many distinct values** (e.g. `userId`) to avoid hot partitions |
| Sort key | Optional; items with the same partition key are sorted by it (e.g. `orderDate`) |
| Capacity modes | **On-demand** (pay per request, no planning) or **Provisioned** (RCU/WCU with auto scaling, cheaper for steady traffic) |

| Concept | Detail |
|---|---|
| Reads | `GetItem` (one item), `Query` (by partition key, efficient), `Scan` (whole table, expensive: avoid) |
| Consistency | Eventually consistent reads by default; strongly consistent option |
| Indexes | **GSI** (different partition/sort key, any time) · **LSI** (same partition key, different sort key, only at creation) |
| Features | TTL (auto-delete expired items), **Streams** (change events → Lambda), PITR backups, Global Tables (multi-region active-active), transactions, DAX (in-memory cache) |

**Design rule:** model DynamoDB around your **access patterns** (the queries you need), not around normalized tables.

## ElastiCache

| | Redis OSS / Valkey | Memcached |
|---|---|---|
| Data structures | Strings, hashes, lists, sets, sorted sets, streams | Simple key-value |
| Persistence / replication | Yes (replicas, Multi-AZ failover, snapshots) | No |
| Use | Sessions, leaderboards, queues, caching, pub/sub | Simple, multi-threaded caching |

**Caching patterns:** *lazy loading / cache-aside* (read cache → on miss read DB and fill cache) and *write-through* (write DB and cache together). Always set a **TTL**.

## ⚠️ Lab: private RDS PostgreSQL reachable only from an app instance

1. Create SG `db-sg` allowing **5432 from** `app-sg` (not from the internet).
2. Create a DB subnet group with two **private** subnets in different AZs.
3. Create RDS PostgreSQL: `db.t4g.micro`, 20 GB gp3, single-AZ (lab), **Publicly accessible: No**, `db-sg`, manage the master password in **Secrets Manager**, backups 1 day.
4. From the app instance: `sudo dnf install -y postgresql16` then `psql -h <db-endpoint> -U postgres -d postgres` → `SELECT version();`.
5. From your laptop the endpoint does **not** connect: correct, it's private.
6. **Cleanup:** delete the DB (skip final snapshot for labs), delete its automated backups, the subnet group, the secret and the SG.

## Database connectivity troubleshooting

| Symptom | Check |
|---|---|
| Connection timeout | SG on the DB allows the port from the **app's SG**? Same VPC (or peering/TGW routes)? NACLs? DB "Available"? |
| `password authentication failed` | Wrong user/password; secret rotated but app cached the old one |
| `could not translate host name` | Wrong endpoint, VPC DNS settings off, private hosted zone not associated |
| `too many connections` | Connection leaks / no pooling → use RDS Proxy or a pool; raise `max_connections` carefully |
| Slow queries | Performance Insights → top SQL; missing indexes; instance too small (CPU/memory/IOPS) |
| Replica data is stale | Replica lag (`ReplicaLag` metric); read your own writes from the writer |

## 7.18 Application integration (decoupling services)

**Why decouple:** if service A calls service B directly and B is slow or down, A fails too. A queue or event bus in between absorbs spikes, enables retries and lets each side scale independently.

### SQS (Simple Queue Service)

```
Producer ──send──► [ SQS queue ] ──poll/receive──► Consumer ──delete when done
                          │ after N failed receives
                          ▼
                  [ Dead-letter queue ]   ← inspect and replay failed messages
```

| Concept | Detail |
|---|---|
| Standard queue | Nearly unlimited throughput, **at-least-once** delivery, best-effort ordering → consumers must be **idempotent** (safe to process twice) |
| FIFO queue | Exactly-once processing, strict order per message group; name ends with `.fifo` |
| Visibility timeout | After a consumer receives a message it's hidden for this time (default 30 s). Not deleted in time → becomes visible again. Set it longer than processing time |
| Retention | Default 4 days, max 14 days |
| Long polling | `WaitTimeSeconds` up to 20 s: fewer empty responses, lower cost |
| Dead-letter queue (DLQ) | Messages that failed `maxReceiveCount` times go here |
| Delay queues | Postpone delivery |
| Common pairing | SQS → Lambda (event source mapping), SQS → ECS/EC2 workers scaled on queue depth |

### SNS (Simple Notification Service)

**Pub/sub:** a publisher sends to a **topic**; every subscriber gets a copy (SQS, Lambda, HTTPS, email, SMS, mobile push). **Fan-out pattern:** SNS topic → several SQS queues so each consumer processes every event independently. Message filtering policies let subscribers receive only matching messages.

### EventBridge

**Serverless event bus:** AWS services, SaaS apps and your own apps publish **events** (JSON); **rules** match patterns and route to **targets** (Lambda, SQS, Step Functions, ECS tasks, API destinations).

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": { "state": ["stopped", "terminated"] }
}
```

Also: **EventBridge Scheduler** (cron/rate schedules, one-time schedules), archive & replay, schema registry, Pipes (point-to-point integrations).

### Step Functions

**Workflow orchestration** as a state machine (JSON/visual): steps, branches, retries with backoff, parallel work, waits, human approval, error handling. Calls Lambda and 200+ AWS services directly. Standard (long-running, exactly-once) vs Express (high-volume, short).

### Streaming

| Service | What | Use |
|---|---|---|
| Kinesis Data Streams | Real-time data streams with shards, ordered per shard, replay | Clickstreams, IoT, logs |
| Amazon Data Firehose | Load streaming data into S3, Redshift, OpenSearch, Splunk with no code | Log/metric delivery |
| MSK | Managed Apache Kafka | Kafka-based event platforms |
| Amazon MQ | Managed ActiveMQ / RabbitMQ | Migrating apps that use standard brokers (JMS, AMQP) |

| Choose | When |
|---|---|
| SQS | One consumer group processes work items (task queue) |
| SNS | Push the same message to many subscribers |
| EventBridge | Route events by content between many services/SaaS; schedules |
| Kinesis / MSK | High-volume ordered streams, multiple readers, replay |
| Step Functions | Multi-step business workflows with state and retries |

## 7.19 Analytics services (awareness level)

| Service | What it does |
|---|---|
| Athena | Run SQL directly on files in S3 (CSV, JSON, Parquet); pay per data scanned. Great for querying logs (ALB, CloudTrail, VPC Flow Logs) |
| Glue | Serverless ETL + **Data Catalog** (table definitions used by Athena/Redshift/EMR) + crawlers |
| EMR | Managed Hadoop/Spark clusters for big data processing |
| Redshift | Data warehouse; Redshift Serverless; Spectrum queries S3 |
| QuickSight | Business intelligence dashboards |
| Lake Formation | Build and secure data lakes on S3 with fine-grained permissions |
| OpenSearch Service | Search and log analytics (ELK-style), dashboards |

**Data lake pattern:** raw data lands in S3 → Glue catalogs/transforms → Athena/Redshift query → QuickSight dashboards.

## 7.20 Serverless: Lambda and API Gateway

### Lambda

**What:** run code **without managing servers**. AWS runs your function when an event arrives and bills per request and per millisecond of execution. **Why:** no patching or capacity planning, scales automatically from zero to thousands of parallel executions, cheap for spiky or low traffic. **In companies:** APIs, file processing (S3 upload → thumbnail), scheduled automation (cleanup, reports, auto-stop instances), event processing (SQS, DynamoDB Streams, EventBridge), glue between services.

| Concept | Detail |
|---|---|
| Runtime | Python, Node.js, Java, .NET, Ruby, Go/Rust (custom runtime), or a **container image** (up to 10 GB) |
| Handler | Function entry point, e.g. `app.handler` → file `app.py`, function `handler(event, context)` |
| Event | JSON input describing what happened (S3 upload, API request, SQS batch…) |
| Execution role | IAM role the function uses to call AWS (least privilege!) |
| Memory | 128 MB to 10,240 MB; **CPU scales with memory** |
| Timeout | Default 3 s, **max 15 minutes** |
| Ephemeral storage | `/tmp`, 512 MB default, up to 10 GB |
| Environment variables | Configuration (not secrets: read secrets from Secrets Manager/Parameter Store) |
| Concurrency | Parallel executions; regional account limit (default 1,000, can be raised). **Reserved concurrency** caps a function; **provisioned concurrency** keeps instances warm |
| Cold start | First request on a new execution environment is slower (init). Reduce with smaller packages, provisioned concurrency, SnapStart (Java/Python/.NET) |
| Versions & aliases | Immutable versions; aliases ( `prod` → v7) enable safe deployments and weighted traffic shifting |
| Layers | Shared libraries packaged separately |
| VPC access | Attach to private subnets to reach RDS etc. (then internet needs a NAT or VPC endpoints) |
| Invocation types | Synchronous (API Gateway), asynchronous (S3, SNS, EventBridge: built-in retries + on-failure destination/DLQ), poll-based (SQS, Kinesis, DynamoDB Streams via event source mappings) |
| Logs | Automatically to CloudWatch Logs group `/aws/lambda/<name>` |

**Example: stop running instances tagged `AutoStop=true` (scheduled nightly by EventBridge Scheduler):**

```
import boto3

ec2 = boto3.client("ec2")

def handler(event, context):
    resp = ec2.describe_instances(Filters=[
        {"Name": "tag:AutoStop", "Values": ["true"]},
        {"Name": "instance-state-name", "Values": ["running"]},
    ])
    ids = [i["InstanceId"] for r in resp["Reservations"] for i in r["Instances"]]
    if ids:
        ec2.stop_instances(InstanceIds=ids)
    print(f"Stopped: {ids}")
    return {"stopped": ids}
```

Execution role needs `ec2:DescribeInstances`, `ec2:StopInstances`, plus `AWSLambdaBasicExecutionRole` for logs.

**Good for Lambda:** short, event-driven, stateless work. **Not good:** long jobs (>15 min), steady very high load (containers may be cheaper), workloads needing local state.

### API Gateway

| Type | Use |
|---|---|
| **HTTP API** | Simple, cheaper, faster; Lambda/HTTP backends, JWT auth. **Default choice** |
| REST API | More features: API keys & usage plans, request validation, caching, WAF, private APIs |
| WebSocket API | Real-time two-way apps (chat) |

Features: stages ( `dev` , `prod` ), custom domains (ACM), throttling, CORS, authorizers (IAM, Cognito, JWT, Lambda), access logs.

**Classic serverless API:** `Client → API Gateway → Lambda → DynamoDB` (+ Cognito for user login).

### Other serverless building blocks

Cognito (user sign-up/sign-in, JWT tokens) · AppSync (managed GraphQL) · SQS/SNS/EventBridge/Step Functions (7.18) · Fargate (serverless containers) · Aurora Serverless v2 · DynamoDB on-demand.

## 7.21 Containers on AWS

| Service | What | When |
|---|---|---|
| **ECR** | Private container image registry | Always: where your images live |
| **ECS** | AWS's own container orchestrator (simpler than Kubernetes) | Most teams that just want to run containers on AWS |
| **EKS** | Managed Kubernetes | Teams standardizing on Kubernetes, multi-cloud skills, rich ecosystem |
| **Fargate** | Serverless compute for ECS and EKS (no EC2 nodes to manage) | Default for small/medium workloads |
| App Runner | Give it a container image or repo → running HTTPS service | Simplest web apps/APIs |
| Lambda (container images) | Run a container image as a function | Event-driven work packaged as images |
| AWS Batch | Batch jobs on managed compute | Large batch processing |

### ECR

| Feature | Detail |
|---|---|
| Repository | One per image name, e.g. `shop/api` |
| Login | `aws ecr get-login-password --region R \| docker login --username AWS --password-stdin <acct>.dkr.ecr.R.amazonaws.com` |
| Push | `docker tag api:1.4.2 <acct>.dkr.ecr.R.amazonaws.com/shop/api:1.4.2` → `docker push ...` |
| **Tag immutability** | Prevents overwriting an existing tag (turn it on) |
| Image scanning | Basic or Enhanced (Inspector) vulnerability scanning on push |
| Lifecycle policies | Delete untagged/old images automatically (saves storage cost) |
| Replication / pull-through cache | Copy images across regions/accounts; cache public images (Docker Hub, ECR Public) |

## ECS in depth

```
ECS CLUSTER
 └─ SERVICE "api" (desired count 3, rolling deploy, behind ALB target group)
      └─ TASKS (running copies of the task definition)
           └─ containers: api:1.4.2 (+ sidecars)
Runs on: FARGATE (serverless)  or  EC2 instances (capacity providers / ASG)
```

| Concept | Detail |
|---|---|
| Task definition | JSON blueprint: image, CPU/memory, ports, env vars, **secrets** (from Secrets Manager/SSM), log config, IAM roles. Versioned (revisions) |
| Task | A running instance of a task definition |
| Service | Keeps N tasks running, replaces failed ones, integrates with ALB/NLB, scales with Application Auto Scaling |
| Launch type / capacity provider | **Fargate** (no servers) or **EC2** (you manage the instances; more control, can be cheaper at scale) |
| **Task role** | Permissions for **your app** inside the container (e.g. read S3) |
| **Task execution role** | Permissions for **ECS itself** to pull the image from ECR, fetch secrets, write logs |
| Networking | `awsvpc` mode: each task gets its own ENI and private IP and its own security group |
| Deployments | Rolling update (min healthy % / max %), deployment circuit breaker with automatic rollback, blue/green via CodeDeploy |
| Service discovery | Cloud Map / Service Connect for service-to-service names |
| Logs | `awslogs` driver → CloudWatch Logs; or FireLens |
| ECS Exec | `aws ecs execute-command` → shell into a running container (debugging) |

**Minimal Fargate task definition (excerpt):**

```
{
  "family": "shop-api",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::123456789012:role/shop-api-task-role",
  "containerDefinitions": [{
    "name": "api",
    "image": "123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api:1.4.2",
    "portMappings": [{ "containerPort": 8000 }],
    "secrets": [{
      "name": "DB_PASSWORD",
      "valueFrom": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:shop/db-AbCdEf"
    }],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/shop-api",
        "awslogs-region": "ap-south-1",
        "awslogs-stream-prefix": "api"
      }
    }
  }]
}
```

### ECS troubleshooting

| Symptom | Cause |
|---|---|
| Task stops immediately | Read **stopped reason** in the console/ `describe-tasks` : `CannotPullContainerError` (image name/tag, ECR permissions on execution role, no route to ECR → NAT or VPC endpoints), `ResourceInitializationError` (can't fetch secrets/logs), app exit code (check logs) |
| Tasks keep restarting | Failing ALB health checks (path/port/SG), container health check failing, out of memory (exit 137) |
| Service stuck "deploying" | New tasks never become healthy → circuit breaker rolls back |

### EKS (overview; details in the Kubernetes notes)

| Topic | Detail |
|---|---|
| Control plane | Managed by AWS across AZs; billed per cluster-hour |
| Data plane | **Managed node groups** (EC2), self-managed nodes, **Fargate profiles**, or **EKS Auto Mode** (AWS manages nodes, scaling and core add-ons) |
| Networking | **VPC CNI**: pods get VPC IPs (plan subnet sizes!) |
| Load balancing | AWS Load Balancer Controller: Ingress → ALB, Service type LoadBalancer → NLB |
| Pod permissions | **EKS Pod Identity** or IRSA (IAM roles for service accounts) |
| Access | Access entries map IAM principals to Kubernetes RBAC; `aws eks update-kubeconfig --name <cluster>` |
| Add-ons | VPC CNI, CoreDNS, kube-proxy, EBS CSI driver, metrics-server; Karpenter for node autoscaling |
| Upgrades | One minor Kubernetes version at a time: control plane → add-ons → nodes |

| ECS vs EKS | ECS | EKS |
|---|---|---|
| Learning curve | Lower | Higher (Kubernetes) |
| Portability | AWS only | Kubernetes anywhere |
| Ecosystem | AWS integrations | Huge Kubernetes ecosystem (Helm, Argo CD, operators) |
| Control-plane cost | Free | Per cluster-hour |

## 7.22 Other compute options

| Service | What | Use |
|---|---|---|
| Elastic Beanstalk | Upload code; it creates EC2, ASG, ALB, monitoring for you (PaaS) | Simple web apps without managing infrastructure details |
| Lightsail | Simple VPS bundles with fixed monthly price | Small websites, blogs, learners |
| AWS Batch | Queue and run batch jobs on EC2/Fargate/Spot | Data processing, rendering |
| Outposts | AWS racks in your data center | Low latency to on-prem systems, data residency |

## 7.23 Management, operations and Infrastructure as Code

### AWS Systems Manager (SSM)

A toolbox for operating fleets of servers (EC2 and on-premises). Requires the **SSM agent** and an instance role with `AmazonSSMManagedInstanceCore`.

| Capability | Use |
|---|---|
| **Session Manager** | Shell access with no SSH keys or open ports; sessions logged to S3/CloudWatch |
| **Run Command** | Run a script on many instances at once (by tag) |
| **Patch Manager** | Scan and install OS patches on a schedule (patch baselines, maintenance windows) |
| **Parameter Store** | Configuration and secrets ( `SecureString` encrypted with KMS); hierarchical names like `/shop/prod/db/host` |
| Automation | Runbooks for common tasks (restart, create AMI, remediate) |
| State Manager | Keep instances in a defined state (install agent, settings) |
| Inventory | Installed software and configuration per instance |
| Fleet Manager | Console view of managed nodes |

```
aws ssm start-session --target i-0123456789abcdef0
aws ssm send-command --document-name AWS-RunShellScript \
  --targets Key=tag:Env,Values=dev --parameters 'commands=["uptime","df -h"]'
aws ssm put-parameter --name /shop/dev/db/password --type SecureString --value '<secret>'
aws ssm get-parameter --name /shop/dev/db/password --with-decryption \
  --query Parameter.Value --output text
```

### CloudFormation and CDK

**CloudFormation** = AWS's native IaC service. You write a **template** (YAML/JSON); CloudFormation creates a **stack** and manages updates, dependencies and rollbacks.

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Versioned S3 bucket
Parameters:
  Env:
    Type: String
    AllowedValues: [dev, prod]
Resources:
  AssetsBucket:
    Type: AWS::S3::Bucket
    Properties:
      VersioningConfiguration:
        Status: Enabled
      Tags:
        - Key: Env
          Value: !Ref Env
Outputs:
  BucketName:
    Value: !Ref AssetsBucket
```

| Concept | Detail |
|---|---|
| Template sections | `Parameters` , `Mappings` , `Conditions` , `Resources` (required), `Outputs` |

| Concept | Detail |
|---|---|
| Intrinsic functions | `!Ref`, `!GetAtt`, `!Sub`, `!Join`, `!If`, `!ImportValue` |
| Stack | Deployed instance of a template; failed creates/updates **roll back** automatically |
| Change set | Preview of changes before updating (like `terraform plan`) |
| Drift detection | Find manual changes made outside CloudFormation |
| StackSets | Deploy a stack to many accounts/regions |
| Nested stacks / modules | Reuse |

**CDK (Cloud Development Kit):** write infrastructure in TypeScript/Python/Java/Go → it generates CloudFormation.

| | Terraform | CloudFormation | CDK |
|---|---|---|---|
| Clouds | Multi-cloud + SaaS providers | AWS only | AWS only (generates CloudFormation) |
| Language | HCL | YAML/JSON | Programming languages |
| State | State file (you manage: S3 + locking) | Managed by AWS | Managed by AWS |
| Typical use | Most DevOps teams, multi-cloud | AWS-centric orgs, AWS service templates | Developer-heavy teams |

### AWS Config

Records the **configuration history** of resources and evaluates **rules** (e.g. "S3 buckets must block public access", "EBS volumes must be encrypted"). Non-compliant resources can be **auto-remediated** with SSM Automation. Conformance packs bundle rules for standards (CIS, PCI).

### Other management services

| Service | Use |
|---|---|
| Service Catalog | Approved, self-service products (pre-built CloudFormation/Terraform) for teams |
| Trusted Advisor | Checks for cost, security, fault tolerance, performance, service limits |
| Service Quotas | View and request increases for limits |
| Health Dashboard | AWS service events affecting your account |
| Resource Groups & Tag Editor | Find and manage resources by tag |
| Compute Optimizer | Right-sizing recommendations for EC2, ASG, EBS, Lambda, ECS |
| License Manager | Track software licenses |

## 7.24 Monitoring and auditing

### CloudWatch

| Feature | Detail |
|---|---|
| **Metrics** | Time-series numbers grouped by **namespace** (`AWS/EC2`, `AWS/RDS`, custom `Shop/App`) and **dimensions** (e.g. `InstanceId`). Standard 1-minute (or 5-min basic for EC2), high-resolution down to 1 s for custom metrics |

| Feature | Detail |
|---|---|
| Statistics | Average, Sum, Minimum, Maximum, SampleCount, percentiles (p90, p99) |
| **Alarms** | Watch a metric over N periods → states `OK`, `ALARM`, `INSUFFICIENT_DATA` → actions: SNS notification, Auto Scaling, EC2 stop/reboot/recover, SSM. **Composite alarms** combine alarms to reduce noise. Anomaly detection bands |
| Dashboards | Graphs and numbers across services/regions |
| **Logs** | Log **groups** (per app) → log **streams** (per instance/container). **Set retention** (default: never expire = growing cost) |
| Metric filters | Turn log patterns into metrics (count `ERROR` lines → alarm) |
| **Logs Insights** | Query language for logs |
| Subscription filters | Stream logs to Lambda, Kinesis, OpenSearch |
| CloudWatch agent | Sends **memory, disk space**, processes and log files from EC2/on-prem (not collected by default) |
| Container Insights / Lambda Insights | Metrics for ECS, EKS, Lambda |
| Synthetics | Canary scripts that test your website/API every few minutes |
| Application Signals / X-Ray | Application performance monitoring and distributed **tracing** |

**Logs Insights examples:**

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

```
stats count(*) as errors by bin(5m)
| filter @message like /Exception/
```

**Create a CPU alarm with email notification:**

```
TOPIC=$(aws sns create-topic --name ops-alerts --query TopicArn --output text)
aws sns subscribe --topic-arn $TOPIC --protocol email \
  --notification-endpoint you@example.com      # then confirm the email
aws cloudwatch put-metric-alarm --alarm-name web-cpu-high \
  --namespace AWS/EC2 --metric-name CPUUtilization --statistic Average \
  --dimensions Name=InstanceId,Value=i-0123456789abcdef0 \
  --period 300 --evaluation-periods 2 --threshold 80 \
  --comparison-operator GreaterThanThreshold --alarm-actions $TOPIC
```

## CloudTrail

| Topic | Detail |
|---|---|
| What | Records **API calls**: who, what, when, from where (user/role, action, source IP, request parameters, result) |
| Event history | Last **90 days** of management events, free, on by default |

| Topic | Detail |
|---|---|
| Trail | Delivers events to S3 (and optionally CloudWatch Logs) for long-term retention; **organization trail** covers all accounts |
| Event types | **Management events** (create/delete/modify resources), **data events** (S3 object reads/writes, Lambda invokes: extra cost), Insights events (unusual API activity) |
| CloudTrail Lake | Store and query events with SQL |
| Protection | Log file integrity validation, S3 bucket in a separate log-archive account, Object Lock |

**Typical questions CloudTrail answers:** "Who deleted this security group?", "Who changed the bucket policy?", "Was the root user used?". Search: CloudTrail → Event history → filter by *Event name* (`DeleteSecurityGroup`) or *Resource name*.

### EventBridge for operations

Rules on events like *EC2 state change*, *GuardDuty finding*, *CloudTrail API call* (e.g. `ConsoleLogin` by root) → SNS/Slack/Lambda for alerts and auto-remediation.

## 7.25 Security services and data protection

### Encryption and secrets

| Service | What | Key points |
|---|---|---|
| **KMS** | Create and control encryption keys | AWS managed keys vs **customer managed keys** (you control key policy, rotation, deletion). **Envelope encryption**: a data key encrypts data; KMS encrypts the data key. Every use is logged in CloudTrail. Key policy + IAM decide who can use a key |
| CloudHSM | Dedicated hardware security modules | Strict compliance needing single-tenant HSMs |
| **Secrets Manager** | Store secrets (DB passwords, API keys) with **automatic rotation** | Apps fetch at runtime via SDK/ECS/EKS integration; paid per secret |
| **Parameter Store** | Config + `SecureString` secrets | Standard tier free; no built-in rotation |
| **ACM** | Free public TLS certificates for ALB, CloudFront, API Gateway, with automatic renewal | Certificates for CloudFront must be in `us-east-1` |

**Encryption at rest:** turn on for EBS (default encryption), S3 (default SSE-S3, or SSE-KMS), RDS, DynamoDB, EFS, backups. **In transit:** HTTPS/TLS everywhere; enforce with bucket policies (`aws:SecureTransport`), ALB HTTPS listeners, RDS `require SSL`.

### Threat detection and protection

| Service | What it does |
|---|---|
| **GuardDuty** | Threat detection using CloudTrail, VPC Flow Logs, DNS logs (+ S3, EKS, RDS, Lambda, runtime monitoring): compromised instances, crypto-mining, credential misuse, unusual API calls |

| Service | What it does |
|---|---|
| **Inspector** | Continuous vulnerability scanning of EC2, ECR images and Lambda (CVEs, network exposure) |
| Macie | Finds sensitive data (personal data, credentials) in S3 |
| **Security Hub** | Central view of findings + automated checks against standards (AWS Foundational Security Best Practices, CIS) |
| Detective | Investigates the root cause of findings (graph of activity) |
| **WAF** | Web application firewall for CloudFront, ALB, API Gateway, AppSync: managed rule groups (OWASP top risks, SQL injection, XSS, bad IPs, bots), rate-based rules |
| **Shield** | DDoS protection: Standard (free, automatic) and Advanced (paid, 24/7 response team, cost protection) |
| Network Firewall | Managed stateful firewall/IPS for VPC traffic (domain filtering, Suricata rules) |
| Firewall Manager | Central management of WAF, Shield, SGs, Network Firewall across accounts |
| IAM Access Analyzer | External access and unused access findings, policy generation and validation |
| Audit Manager / Artifact | Compliance evidence / AWS compliance reports (SOC, ISO, PCI) |

**Incident response basics (compromised instance):** isolate (replace SG with a no-access SG), snapshot volumes for forensics, revoke/rotate credentials (role sessions, keys), investigate with CloudTrail/GuardDuty/Detective, rebuild from a clean AMI, write a postmortem.

## 7.26 Multi-account governance

```
AWS Organizations (management account: billing + governance only, no workloads)
├── OU Security        ── Log Archive account (CloudTrail, Config logs)
│                         Security Tooling account (GuardDuty, Security Hub admin)
├── OU Infrastructure  ── Network account (Transit Gateway, shared VPCs, DNS)
│                         Shared Services account (CI/CD, ECR)
├── OU Workloads
│     ├── OU Dev   ── shop-dev account
│     └── OU Prod  ── shop-prod account
└── OU Sandbox     ── experiment accounts (strict budgets)
```

| Service / concept | Detail |
|---|---|
| **Organizations** | Groups accounts, **consolidated billing** (one bill, combined volume discounts), organizational units (OUs) |
| **SCPs** | Maximum permissions for accounts/OUs: e.g. deny leaving the organization, deny disabling CloudTrail/GuardDuty, allow only approved regions. SCPs never grant permissions |
| RCPs | Resource control policies: org-wide limits on who can access your resources |
| Tag / backup / AI opt-out policies | Org-wide standards |
| **Control Tower** | Automated landing zone: account factory, guardrails (preventive SCPs + detective Config rules), central logging |
| **IAM Identity Center** | Single sign-on across all accounts with permission sets |

| Service / concept | Detail |
|---|---|
| RAM (Resource Access Manager) | Share subnets, TGWs, Route 53 Resolver rules, etc. across accounts |

**Why multi-account:** strongest isolation boundary (blast radius), clear cost per team/environment, separate quotas, simpler permissions.

**Example SCP: allow only two regions (global services exempted):**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOtherRegions",
    "Effect": "Deny",
    "NotAction": [
      "iam:*", "organizations:*", "route53:*", "cloudfront:*",
      "support:*", "sts:*", "budgets:*"
    ],
    "Resource": "*",
    "Condition": { "StringNotEquals": { "aws:RequestedRegion": ["ap-south-1", "us-east-1"] } }
  }]
}
```

## 7.27 Cost management (FinOps basics)

| Tool | Use |
|---|---|
| Billing dashboard / Bills | What you were charged, per service and region |
| **Cost Explorer** | Explore and forecast cost by service, account, tag, usage type; RI/Savings Plans recommendations |
| **Budgets** | Alerts (and actions) when cost/usage/coverage crosses thresholds |
| Cost Anomaly Detection | ML alerts for unusual spend |
| Cost and Usage Report (Data Exports) | Line-item billing data to S3 for Athena/QuickSight analysis |
| Cost allocation tags | Activate tags (e.g. `Project`) to split costs |
| Compute Optimizer / Trusted Advisor | Right-sizing and idle-resource findings |
| Pricing Calculator | Estimate before you build |

**Cost optimization checklist:** 1. **Delete idle resources:** unattached EBS volumes, old snapshots/AMIs, unused EIPs/public IPv4, idle load balancers, stopped instances you don't need, forgotten NAT Gateways. 2. **Right-size** instances (Compute Optimizer), move to **Graviton**, gp2 → gp3. 3. **Schedule** dev/test to stop nights and weekends. 4. **Commit** for steady usage (Savings Plans); **Spot** for flexible work. 5. **Storage lifecycle:** S3 lifecycle rules/Intelligent-Tiering, log retention in CloudWatch. 6. **Network:** VPC gateway endpoints for S3/DynamoDB instead of NAT, keep chatty traffic in one AZ, use CloudFront to cut data transfer out. 7. **Tag everything** and review Cost Explorer weekly.

## 7.28 High availability, backup and disaster recovery

### Availability building blocks

| Layer | HA approach |
|---|---|
| DNS | Route 53 (100% SLA), health checks, failover routing |
| Edge | CloudFront, Global Accelerator |
| Load balancing | ALB/NLB across ≥2 AZs |
| Compute | Auto Scaling Group / ECS service / EKS across ≥2 AZs, stateless instances |
| Database | RDS Multi-AZ / Aurora replicas / DynamoDB (multi-AZ by default) |
| Storage | S3 (multi-AZ by default), EFS regional, EBS snapshots |
| Network | One NAT Gateway per AZ, redundant VPN tunnels / DX + VPN backup |

**Design for failure:** assume any instance, AZ, or dependency can fail; use health checks, retries with backoff, timeouts, idempotency and graceful degradation.

### Backup

**AWS Backup:** central backup **plans** (schedule + retention + lifecycle to cold storage), **vaults** (with Vault Lock for immutability), cross-region and cross-account copies, for EC2/EBS, RDS/Aurora, DynamoDB, EFS, FSx, S3 and more. **Test restores regularly**: an untested backup is not a backup.

### Disaster recovery strategies

| Strategy | Idea | RTO | RPO | Cost |
|---|---|---|---|---|
| Backup & restore | Back up to another region; rebuild with IaC when needed | Hours | Hours (last backup) | $ |
| Pilot light | Core data replicated live (DB replica); servers off/minimal | Tens of minutes | Minutes | $$ |
| Warm standby | Scaled-down full copy always running in the DR region | Minutes | Seconds–minutes | $$$ |
| Multi-site active/active | Full capacity in 2+ regions serving traffic | Near zero | Near zero | $$$$ |

- **RPO (Recovery Point Objective):** how much data loss is acceptable (time since last good copy).
- **RTO (Recovery Time Objective):** how long the system may be down.
- **Elastic Disaster Recovery (DRS):** continuous block-level replication of servers into AWS for fast recovery.
- IaC (Terraform/CloudFormation) makes rebuilding another region repeatable.

## 7.29 Migration to AWS (awareness)

**The 7 Rs:** Retire · Retain · **Rehost** (lift-and-shift) · **Relocate** (e.g. VMware Cloud on AWS) · **Replatform** (lift-tinker-shift, e.g. to RDS) · Repurchase (move to SaaS) · **Refactor** (re-architect, e.g. to containers/serverless).

| Service | Use |
|---|---|
| Migration Hub / Application Discovery Service | Inventory and track migrations |

| Service | Use |
|---|---|
| Application Migration Service (MGN) | Rehost servers into EC2 with continuous replication |
| Database Migration Service (DMS) + Schema Conversion | Migrate databases (same or different engine) with minimal downtime |
| DataSync | Copy large file data (NFS/SMB/S3) online |
| Storage Gateway | Hybrid storage: on-prem apps use S3/backups through a local gateway |
| Snow family | Physical devices to ship large data offline |
| Transfer Family | Managed SFTP/FTPS/FTP into S3/EFS |

## 7.30 Well-Architected Framework

| Pillar | Key practices |
|---|---|
| **Operational Excellence** | IaC, small frequent reversible changes, runbooks, observability, learn from failures |
| **Security** | Strong identity foundation (least privilege, SSO, roles), traceability (CloudTrail), protect data (encryption), security at all layers, automate, prepare for incidents |
| **Reliability** | Multi-AZ, auto-recovery, scale horizontally, test recovery, manage quotas, backups |
| **Performance Efficiency** | Right resource types, serverless/managed services, measure and experiment, caching |
| **Cost Optimization** | Pay only for what you need, measure (tags, Cost Explorer), right-size, commit/Spot, managed services |
| **Sustainability** | Maximize utilization, efficient hardware (Graviton), reduce data movement and storage |

The **Well-Architected Tool** in the console asks review questions per workload and tracks improvement plans.

## 7.31 Reference architectures

### A. Highly available 3-tier web application

```
Users → Route 53 → CloudFront (+ WAF) → ALB (public subnets, 2 AZs)
                                          |
                      ASG / ECS service (private app subnets, 2 AZs)
                        |             |              |
            ElastiCache (sessions)  S3 (assets)   Secrets Manager (DB creds)
                        |
                RDS Multi-AZ / Aurora (private data subnets)
Logs/metrics → CloudWatch · audit → CloudTrail · backups → AWS Backup
```

### B. Static website

`Route 53 → CloudFront (ACM certificate, OAC) → private S3 bucket`. Cheap, fast, no servers. Deploy with `aws s3 sync` + CloudFront invalidation from CI.

### C. Serverless API

`Route 53 → API Gateway (HTTP API, JWT authorizer with Cognito) → Lambda → DynamoDB`, async work through SQS/EventBridge → Lambda, monitoring with CloudWatch + X-Ray.

### D. Containers platform

`GitHub Actions (OIDC) → ECR → ECS Fargate or EKS (private subnets) behind ALB`, secrets from Secrets Manager, logs to CloudWatch, Terraform for all infrastructure.

### E. Event-driven processing

`S3 upload → EventBridge rule → Step Functions (validate → process with Lambda/Fargate → store in DynamoDB → notify with SNS)`, failures to a DLQ.

## 7.32 AWS troubleshooting playbook

| Problem | Investigation order |
|---|---|
| Anything returns **AccessDenied** | `aws sts get-caller-identity` → error message (action + resource) → identity policies → resource policies (bucket/KMS) → SCP/boundary → conditions → CloudTrail event |
| **EC2 unreachable** | Status checks → public IP/EIP → SG → NACL → route table (IGW) → OS firewall/service → key/username → use SSM / serial console |
| **Website returns 5xx through ALB** | Target group health (reason codes) → app logs → target SG allows ALB SG → health check path → idle timeouts (504) |
| **Private instance can't reach the internet / AWS APIs** | Route to NAT (in a public subnet) or VPC endpoints → SG/NACL outbound → DNS settings |
| **App can't connect to RDS** | DB SG allows app SG on DB port → same VPC/routes → endpoint name → credentials/secret → DB status/connections |
| **Lambda failing** | CloudWatch Logs for the function → timeout/memory → execution role permissions → VPC access (NAT/endpoints) → throttling (concurrency) |
| **ECS task stopped** | Stopped reason → image pull (execution role, network to ECR) → secrets → health checks → logs → exit code (137 = OOM) |
| **S3 403** | IAM vs bucket policy vs Block Public Access vs KMS key policy → `ListBucket` on bucket ARN vs `GetObject` on `/*` |
| **Unexpected bill** | Cost Explorer by service and **region** → usage type (NAT bytes, data transfer, EBS snapshots, public IPv4) → find and delete; add budgets |
| **Can't launch instances** | Activity/error message → service quotas (vCPU) → capacity in AZ → subnet free IPs → AMI/permissions |

## 7.33 Interview questions and answers

| # | Question | Answer |
|---|---|---|
| 1 | What is the difference between a Region and an AZ? | A region is a geographic area; an AZ is one or more isolated data centers inside a region. Deploy across multiple AZs for HA |
| 2 | Explain the shared responsibility model | AWS secures the cloud (hardware, facilities, managed-service software); customers secure what they put in it (data, IAM, OS patching on EC2, network rules) |
| 3 | IAM user vs role? | User = long-term identity with password/keys. Role = assumed identity with temporary credentials (STS); used by services, CI, federation, cross-account |
| 4 | How are IAM policies evaluated? | Default deny → explicit allow needed → explicit deny always wins; SCPs and permission boundaries limit the maximum |

| # | Question | Answer |
|---|---|---|
| 5 | How does an application on EC2 get AWS credentials safely? | IAM role via instance profile; SDK reads temporary credentials from IMDSv2; never store keys on the instance |
| 6 | Security group vs NACL? | SG: stateful, instance-level, allow-only, all rules evaluated. NACL: stateless, subnet-level, allow+deny, ordered rules, needs ephemeral ports |
| 7 | Public vs private subnet? | Public has a route `0.0.0.0/0 → IGW`; private doesn't (uses NAT for outbound) |
| 8 | How does a private instance download updates? | Through a NAT Gateway in a public subnet (or VPC endpoints for AWS services) |
| 9 | Internet Gateway vs NAT Gateway? | IGW enables two-way internet for public IPs; NAT GW gives outbound-only internet to private subnets |
| 10 | What are VPC endpoints? | Private access to AWS services without internet: gateway (S3/DynamoDB, free, route table) and interface (PrivateLink ENIs, paid) |
| 11 | VPC peering vs Transit Gateway? | Peering: 1-to-1, non-transitive. TGW: hub for many VPCs/VPNs with transitive routing |
| 12 | ALB vs NLB? | ALB: layer 7 HTTP routing (paths/hosts), WAF. NLB: layer 4 TCP/UDP, extreme performance, static IPs, preserves client IP |
| 13 | ALB returns 503. Why? | No healthy targets registered in the target group |
| 14 | How does Auto Scaling know when to scale? | Scaling policies on CloudWatch metrics (target tracking e.g. CPU 50%), schedules, or predictive scaling |
| 15 | EBS vs EFS vs S3 vs instance store? | Block disk for one instance (AZ) / shared NFS across AZs / object storage via API / ephemeral host disk |
| 16 | How do you back up EBS? | Snapshots (incremental, stored in S3), automated by DLM or AWS Backup, copied cross-region for DR |
| 17 | S3 storage classes and lifecycle? | Standard → IA → Glacier tiers by access frequency; lifecycle rules transition/expire automatically |
| 18 | How do you secure an S3 bucket? | Block Public Access, least-privilege IAM, bucket policy (HTTPS only, specific principals), encryption (SSE-KMS), versioning, access logs, CloudFront OAC for public content |
| 19 | RDS Multi-AZ vs read replica? | Multi-AZ = synchronous standby for automatic failover (HA); read replica = async copy for read scaling |
| 20 | When would you use DynamoDB instead of RDS? | Key-value access at massive scale, predictable single-digit-ms latency, serverless scaling, flexible schema |
| 21 | SQS vs SNS vs EventBridge? | Queue for work items (pull) / pub-sub fan-out (push) / content-based event routing with schedules and SaaS sources |
| 22 | What are Lambda limits you must remember? | 15-min max timeout, up to 10 GB memory, concurrency limits, cold starts, stateless |
| 23 | ECS task role vs execution role? | Task role = permissions for the app; execution role = permissions for ECS to pull images, fetch secrets, write logs |
| 24 | ECS vs EKS? | ECS simpler and AWS-native; EKS is Kubernetes with its ecosystem and portability, more complex and costs per cluster |
| 25 | CloudWatch vs CloudTrail? | CloudWatch = performance metrics, logs, alarms. CloudTrail = audit log of API calls (who did what) |

| # | Question | Answer |
|---|---|---|
| 26 | How do you store secrets on AWS? | Secrets Manager (rotation) or SSM Parameter Store SecureString, accessed by IAM roles at runtime, encrypted with KMS |
| 27 | What is KMS envelope encryption? | Data encrypted with a data key; the data key is encrypted by a KMS key; only the encrypted data key is stored |
| 28 | How do you design for high availability? | Multi-AZ for every tier, load balancer + auto scaling, managed multi-AZ databases, health checks, stateless apps, backups |
| 29 | RPO vs RTO; DR strategies? | Data-loss tolerance vs downtime tolerance. Backup/restore → pilot light → warm standby → active/active (cost rises, RTO/RPO fall) |
| 30 | How do you reduce AWS costs? | Delete idle resources, right-size, Graviton, Savings Plans/Spot, schedules, S3 lifecycle, VPC endpoints vs NAT, tags + budgets |
| 31 | Why use multiple AWS accounts? | Isolation/blast radius, separate billing and quotas, simpler permissions; managed with Organizations, SCPs, Control Tower, SSO |
| 32 | What is an SCP? | Organization policy that sets the maximum permissions for accounts; it never grants access |
| 33 | How does CI/CD deploy to AWS without stored keys? | OIDC federation: the pipeline assumes an IAM role via `AssumeRoleWithWebIdentity` with conditions on repo/branch |
| 34 | What's IMDSv2 and why require it? | Session-token based metadata access; blocks SSRF attacks from stealing role credentials |
| 35 | An EC2 instance's public IP changed after restart. Why, and fix? | Auto-assigned public IPs are released on stop; use an Elastic IP or (better) a load balancer/DNS name |
| 36 | CloudFront with a private S3 bucket? | Origin Access Control: bucket policy allows only the distribution; Block Public Access stays on |
| 37 | Terraform vs CloudFormation? | Terraform multi-cloud with its own state file; CloudFormation AWS-native with managed state, change sets and drift detection |
| 38 | Someone deleted a production resource. How do you find out who? | CloudTrail event history / trail: filter by event name and resource; then add guardrails (SCP, IAM, deletion protection) |
| 39 | What happens to data on instance store when you stop the instance? | It is lost (survives only reboots) |
| 40 | How do you give on-prem servers private access to AWS? | Site-to-Site VPN (quick) or Direct Connect (dedicated, consistent), usually via Transit Gateway; DNS via Route 53 Resolver endpoints |

## 7.34 Self-check: I can…

- [ ] Explain regions, AZs, edge locations and the shared responsibility model.
- [ ] Secure a new account: root MFA, admin via SSO/IAM, budgets, CloudTrail.
- [ ] Read and write IAM policies; explain roles, trust policies and STS.
- [ ] Use the AWS CLI with profiles, `--query` and `--filters`.
- [ ] Launch, connect to (SSH and SSM), bootstrap and troubleshoot an EC2 instance.
- [ ] Choose between EBS, EFS, instance store and S3; attach, mount and snapshot a volume.
- [ ] Create a secure S3 bucket with versioning, lifecycle and least-privilege access.
- [ ] Design and build a multi-AZ VPC with public/private subnets, IGW, NAT and endpoints.
- [ ] Explain SGs vs NACLs and debug connectivity with Flow Logs / Reachability Analyzer.

- [ ] Put an ALB in front of an Auto Scaling Group and explain 502/503/504.
- [ ] Configure Route 53 records and routing policies; explain alias vs CNAME.
- [ ] Create a private RDS database with Multi-AZ and backups, and connect from an app.
- [ ] Explain DynamoDB keys, SQS/SNS/EventBridge, Lambda and API Gateway.
- [ ] Explain ECR, ECS (task/service/roles) and EKS at a high level.
- [ ] Set up CloudWatch alarms, log retention, Logs Insights queries and CloudTrail investigations.
- [ ] Explain KMS, Secrets Manager, GuardDuty, WAF and Security Hub.
- [ ] Describe a multi-account setup with Organizations, SCPs and IAM Identity Center.
- [ ] Find and cut costs; set budgets; clean up after every lab.
- [ ] Explain HA patterns, AWS Backup, RPO/RTO and the four DR strategies.
- [ ] Draw and explain the 3-tier, static site, serverless and container reference architectures.

**REMEMBER**

- **Public subnet** = route to an Internet Gateway. Private servers reach the internet through a **NAT Gateway**.
- **Security Group**: stateful, allow-only, per instance. **NACL**: stateless, allow + deny, per subnet.
- First command when anything fails: `aws sts get-caller-identity` (who am I, which account?).
- Well-Architected pillars: Operations, Security, Reliability, Performance, Cost, Sustainability.
- After every lab: **delete resources and check Billing**.